

# Learning Only What Matters Most: Efficient Cross-Modal Attention Training

Rui Cao  
Hong Kong Baptist University  
csrcao@comp.hkbu.edu.hk

Rundong Zuo  
Hong Kong Baptist University  
csrdzuo@comp.hkbu.edu.hk

Guozhong Li  
KAUST  
guozhong.li@kaust.edu.sa

Byron Choi  
Hong Kong Baptist University  
choi@hkbu.edu.hk

Yafei Li  
Zhengzhou University  
ieyfli@zzu.edu.cn

Renchi Yang  
Hong Kong Baptist University  
renchi@hkbu.edu.hk

## ABSTRACT

Transformers have demonstrated strong performance across diverse domains and show increasing promise in multimodal learning. However, scaling to multimodal inputs is computationally costly due to the quadratic complexity of self and cross-modal attention. Existing efficient attention methods, largely designed for unimodal settings, fail to account for modality-specific characteristics. In this paper, we propose the first efficient cross-modal attention mechanism, *LEarning Only what matters most* (LEO), which exploits sparsity in high-dimensional multimodal representations to achieve linearithmic complexity. Specifically, we introduce (1) a lightweight Key matrix column selection with theoretical error bounds, (2) Query matrix pruning guided by modality importance, and (3) attention approximation with histograms to preserve the original distribution with a bounded error. Experiments on multimodal time series forecasting benchmarks show that (LEO) achieves up to 36% end-to-end speedup over vanilla attention and 20% over existing sparse methods, while achieving superior prediction accuracy.

## PVLDB Reference Format:

Rui Cao, Rundong Zuo, Guozhong Li, Byron Choi, Yafei Li, and Renchi Yang. Learning Only What Matters Most: Efficient Cross-Modal Attention Training. PVLDB, 14(1): XXX-XXX, 2020.  
doi:XX.XX/XXX.XX

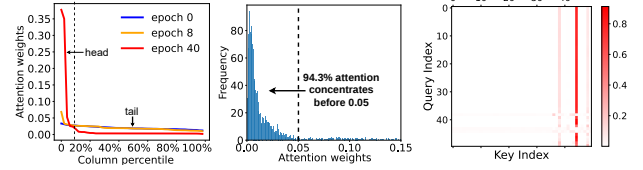
## PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/csrcao/LEO.git>.

## 1 INTRODUCTION

Transformers have significantly advanced artificial intelligence across domains, including natural language processing [56, 60], computer vision [68], and time series analysis [50, 76], among others. Recently, transformers have further emerged as powerful backbones for multimodal learning [74], fueling state-of-the-art performance in tasks such as information retrieval [22], recommendation systems [57], and speech recognition [24]. In this context, cross-modal transformers play a central role by enabling cross-modal interactions and fusion [59]. This paper focuses on multimodal time

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.  
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX



(a) Ranked attention weights by the column mean at various epochs (b) Attention weights distribution just before the model converges (c) Attention weights heatmap on epoch 40

**Figure 1: (a, b) Long-tailed distribution of an attention matrix and (c) cross-modal attention patterns of column-wise attention in various epochs using MulT on CMU-MOSI dataset**

series forecasting [29, 72], a representative multimodal task, where cross-modal attention allows Queries from one modality to attend to tokens in another, aligning them within a shared feature space.

Unimodal transformers [49, 70] suffer from the well-known quadratic complexity  $O(N^2)$  due to self-attention. This limitation evidently extends to cross-modal transformers, where the cost is  $O(MN)$  and becomes  $O(N^2)$  when the modalities have comparable lengths  $M, N$  [48]. Although prior work has improved unimodal attention efficiency through pattern-based [51], kernel-based [75], and sparsity-based methods [5], these approaches are not tailored to multimodal settings. Existing efforts to accelerate cross-modal attention mainly rely on knowledge distillation [52] or token pruning [69] outside the attention module, which reduces input size, but still retains quadratic complexity.

**Opportunities.** Cross-modal attention has several opportunities to improve its computational efficiency while maintaining accuracy.

(1) *Skewed attention distribution.* Attention weights often exhibit a skewed distribution, where only a small fraction of informative token interactions dominate. As shown in Figure 1(a), attention weights transition from nearly uniform (at epochs 0-8) to sharply peaked (at epoch 40), concentrating on a few informative tokens over training. Figure 1(b) further shows that a large proportion of attention weights are below 0.05 at convergence. Importantly, most token pairs, as is evident by the skew long-tailed distribution of attention weights, contribute little to the output (final representation), but they still participate in the attention computation.

(2) *High column-wise attention concentration.* As discovered by previous studies of transformers (e.g., [5]), most attention weights are

localized in a small subset of Key vectors (column vectors) in Figure 1(c). This indicates that Key vectors with a high concentration of attention weights can approximate the attention weights.

(3) *Weak modalities*. Cross-modal attention often exhibits modality imbalance, where one dominant modality can govern attention, while other modalities contribute little to the attention, providing little or redundant information (e.g., [32, 53]). For example, our empirical evidence in Figure 4 shows that pruning the Query inputs can have little impact on the representation, but is more efficient.

**Our approach.** In this paper, we propose the first efficient cross-modal attention mechanism, *LEarning Only what matters most*, called LEO (illustrated in Figure 3), which achieves the lowest KL divergence to vanilla attention weights validated by both theoretical analysis and empirical study on approximation quality (highlight in Figure 2).

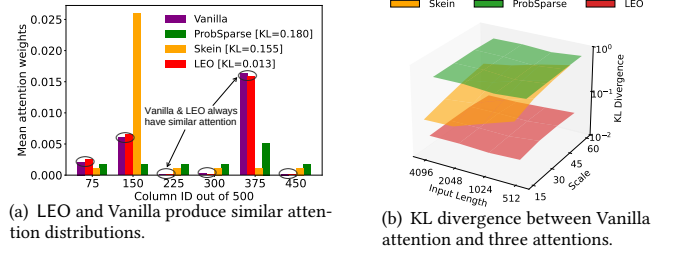
First, LEO introduces a lightweight Key matrix simplification for attention weights, addressing the sparsity in attention, based on the column selection problem. In particular, the column vectors derived from the Key matrix can be selected to form a submatrix that generates most attention weights of the original attention, as an approximation with an error bound. This approach is motivated by the highly skewed nature of attention distributions, in which a small number of elements contribute substantially to the output, whereas most have a negligible impact.

Second, we further optimize the Query matrix of attention, which employs a Query-dependent gating mechanism and prunes the Query input with lower gate scores, acting as an attention estimator with an error bound. To fit the attention across modalities, we also introduce a mechanism to adaptively update the pruning ratio of the Query matrix during computation iterations, based on our estimated modality importance with theoretical analysis.

Third, we introduce a near equal-depth histogram-based approximation of the distributional tail [43]. Specifically, the tail is partitioned into approximate histogram buckets, each represented by its mean to summarize the attention weights over a subset of columns. After building a bucket-mean lookup table on sampled true attention weights and aligning them with an ANN index over the distributional head, our method efficiently retrieves a histogram approximation of attention with a theoretical bound on the KL divergence. A lightweight filtering mechanism is further applied to identify query vectors that benefit the most from the approximate histograms. A cost model is proposed to jointly determine the filtering threshold and the column subset size, enabling tuning the trade-off between efficiency and approximation quality.

As a result, LEO improves computational efficiency by reducing both Query–Key and attention–Value multiplications, while providing the corresponding error bounds for each approximation. We achieve the time complexity of  $\Theta(N(\log N))$  when we assume  $M = \Theta(N)$ , where  $M$  and  $N$  are the input lengths of the Query matrix and Key/Value matrix, respectively. Additionally, our mechanism can be readily integrated into the training of existing cross-modal transformer-based architectures.

Finally, we conduct extensive experiments on the four longest multimodal time series datasets tested in the supervised Aurora framework [54]. The result shows that the Aurora with our attention is about 17%-36% faster than that with the vanilla attention and



**Figure 2: Illustration of attention distribution with attention weights (left) and KL divergence (right) between Vanilla cross-modal attention [49] and the other three attention distributions — the lower the closer to Vanilla attention. Scale is the constant factor independent of input length.**

8%-20% faster than the other compared sparse attention methods operated on the Query or Key (Value) matrix, while achieving the most accurate and robust result.

**Contributions.** In this paper, we make the following contributions:

- We identify and analyze the inefficiency of vanilla cross-modal attention and systematically analyze some properties of cross-modal attention.
- We propose an efficient cross-modal attention mechanism, LEO, that leverages the sparsity and prunes both Query and Key matrices in Sections 4 and 5 with theoretical analysis, respectively.
- We introduce approximation to fit the attention distribution in Section 6, reducing computation while preserving accuracy, validated by both theoretical bounds and our experiments.
- We implement this mechanism within a SOTA multimodal time series forecasting framework, Aurora [54], and evaluate it across four large real-world datasets. Experimental results demonstrate that the framework with our method achieves up to 36% improvement in speed and up to 12% improvement in prediction MSE over that with vanilla cross-modal attention.

## 2 BACKGROUND

In this section, we present the preliminaries and notation. Table 1 summarizes the main symbols.

**Cross-modal Attention.** Self-attention [49] models dependencies within a single modality by projecting inputs into Query, Key, and Value matrices. Cross-modal attention [22] extends this mechanism to multiple modalities, where the Query matrix originates from one modality and the Key/Value matrices from another. As a mechanism for cross-modal fusion, cross-modal attention can be applied at different stages: *early fusion* directly combines inputs from different modalities, whereas *intermediate fusion* first encodes each modality independently and introduces cross-modal interactions at intermediate layers. More details are provided in Section 8. Here, we illustrate the detailed structure of a vanilla cross-modal attention.

Let  $X_a \in \mathbb{R}^{M \times d_a}$ ,  $X_b \in \mathbb{R}^{N \times d_b}$  be their respective input embeddings. The linear projections are defined as:  $Q_a = X_a W_{Q_a}$ ,  $K_b = X_b W_{K_b}$ , and  $V_b = X_b W_{V_b}$ .  $Q_a$  is Query matrix of modality A,  $K_b$  and  $V_b$  are Key and Value modalities of modality B,  $W_{Q_a} \in \mathbb{R}^{d_a \times d}$ ,  $W_{K_b} \in \mathbb{R}^{d_b \times d}$ , and  $W_{V_b} \in \mathbb{R}^{d_b \times d}$  are learnable parameters.

The cross-modal attention output from modality  $B$  to modality  $A$  is defined as:

$$Y_a = CM_{b \rightarrow a}(X_a, X_b) = \text{softmax}\left(\frac{Q_a K_b^\top}{\sqrt{d}}\right) V_b, \quad (1)$$

where the softmax function is applied *row-wise*.

The  $(i, j)$ -th entry of  $\text{softmax}\left(\frac{Q_a K_b^\top}{\sqrt{d}}\right)$  represents the attention weight assigned by the  $i$ -th token in modality  $A$  to the  $j$ -th token in modality  $B$ . The resulting representation  $Y_a \in \mathbb{R}^{M \times d_v}$  preserves the sequence length of modality  $A$  while incorporating information from modality  $B$ .

**Efficiency problem in cross-modal attention.** The dominant cost arises from matrix multiplication and softmax, which requires  $O(MNd_k)$  operations. Since typically  $d_k \ll M, N$ , the complexity is commonly expressed as  $O(MN)$ . When  $M$  is  $\Theta(N)$ , cross-modal attention scales quadratically with respect to the sequence lengths of the two modalities. Multimodal models with cross-modal attention have a strong dependency on the ability to jointly process multimodal information [59]. Therefore, the efficiency problem of cross-modal attention in multimodal tasks has become an urgent issue to address, as more modalities result in increased model complexity and computational needs.

**Sampling-based CSS.** Matrix sketching techniques approximate a large matrix by constructing a compact representation that preserves its dominant structure [19, 62]. Among existing approaches, sampling-based methods are computationally efficient and particularly suitable for large-scale systems. A representative formulation is the *Column Subset Selection* (CSS) problem [4].

**Definition 2.1 (Column Subset Selection).** Given a matrix  $A \in \mathbb{R}^{n \times d}$  and an integer  $k$ , find a subset of column IDs  $S \subseteq \{1, \dots, d\}$  with  $|S| = k$  such that

$$\arg \min_S \|A - P_S A\|_F^2, \quad (2)$$

where  $P_S$  denotes the projection matrix onto the column span of  $A_{:,S}$  and  $\|\cdot\|_F$  is the Frobenius norm.

The objective measures how well the selected columns approximate the column space of  $A$ . Unlike generic low-rank factorization, CSS selects actual columns of the input matrix, thereby preserving structural characteristics while reducing dimensionality. The CSS problem is NP-complete [42], but polynomial-time approximation algorithms with approximation ratio  $O(k+1)$  [11, 18] have been proposed. Practical methods typically rely on sampling [11, 36] or greedy strategies [3, 15] and scale to matrices with large numbers of columns. In cross-modal attention, the matrix  $Q_a K_b^\top \in \mathbb{R}^{M \times N}$  can be viewed as a product involving all columns of  $K_b$  with  $O(MN)$  time complexity. By selecting a subset of columns from  $K_b$  using sampling-based CSS, we approximate the full attention matrix while significantly reducing computation.

**Approximate histograms.** Approximate histograms are used to summarize the distribution of data when processing all raw data is impractical [21]. Traditional histograms require access to all data and become inefficient in large-scale or streaming environments. To address this limitation, approximate histograms maintain a limited number of bins and update them dynamically as new data arrives [1]. By adjusting bins when necessary, they provide a compact representation of the underlying data distribution while using bounded memory. This approach is widely used in database

**Table 1: Summary of frequently used notations**

| Notation                                 | Description                                                        |
|------------------------------------------|--------------------------------------------------------------------|
| $A, B$                                   | Two input modalities                                               |
| $X_a, X_b$                               | Input embeddings of modality $A, B$                                |
| $M, N$                                   | Sequence lengths of modalities $A$ and $B$                         |
| $d$                                      | Dimensions of Query, Key, and Value matrices                       |
| $Q_a$                                    | Query matrix derived from $X_a$                                    |
| $K_b, V_b$                               | Key/Value matrix derived from $X_b$                                |
| $QK^\top$                                | Attention logits before softmax multiplying Query and Key matrix   |
| $Y_a$                                    | Cross-modal attention output from $B$ to $A$                       |
| $A_{\text{sample}}$                      | Query sampled attention matrix                                     |
| $A_{\text{pre}}$                         | Preliminary attention for searching                                |
| $A_{\text{appx}}$                        | Approximate attention after normalization                          |
| $\ \cdot\ _F$                            | Frobenius norm                                                     |
| $I_\mu, I_\nu$                           | Sets of sampled Query row IDs and selected Key $K^\top$ column IDs |
| $Q_{I_\mu}, (K^\top)_{I_\nu}, V_{I_\nu}$ | Matrices consisting of vectors with IDs in $I_\mu, I_\nu$          |
| $\text{sort}(x, \text{desc})$            | Vector $x$ whose elements are sorted in descending order           |

systems [20, 47], streaming analytics [16, 17], and query optimization [12, 64], where approximate statistics are sufficient to estimate data characteristics such as value distribution.

### 3 ANALYSIS AND SOLUTION OVERVIEW

In this section, we present an analysis of cross-modal attention and an overview of our solution, aiming to identify patterns that are specific to cross-modal interactions. Figure 1 shows the two phenomena we observed in cross-modal attention, which can also be observed in unimodal self-attention in our experiments. The  $x$ -axis column percentile refers to the relative rank (expressed as a percentage) of a column after sorting all columns in descending order by their attention weights.

#### 3.1 Long-tailed Distribution in Attention Rows

It is known that when large input matrices have a non-uniform distribution, the vector multiplication and softmax operations tend to amplify the differences among the attention logits before softmax, forming a power-law distribution in the **tail** of the attention distribution [40]. In contrast, those attention weights that contribute more to overall attention are termed the **head** of the distribution. This long-tail distribution has been widely used for efficient attention [5, 73]. To explain it, we propose Theorem 3.1 to discuss the property of the softmax function.

**THEOREM 3.1 (SOFTMAX SUPPRESSION OF SMALL WEIGHTS).** Assume an attention logit matrix before softmax  $e = [e_1, \dots, e_M]^\top \in \mathbb{R}^{M \times N}$ . Let  $e_i \in \mathbb{R}^{1 \times N}$  be the  $i$ -th row of  $e$  and  $A_i = \text{softmax}(e_i)$ . Assume the logits  $e_{ij}$  are uniformly bounded: there exists a constant  $\zeta > 0$ , such that  $|e_{ij}| \leq \zeta$ ,  $\forall j \in \{1, \dots, N\}$ . Then,

$$\lim_{N \rightarrow \infty} \min_{1 \leq j \leq N} a_{ij} = 0. \quad (3)$$

Theorem 3.1 formalizes a property of softmax, leading to a long-tailed distribution in attention weights, wherein a small subset of informative tokens contributes significantly to the overall attention distribution, and most tokens have negligible attention. This explains the skewed attention distribution in Figure 1(a)-(b).

#### 3.2 High Column-wise Attention Concentration

In addition to the long-tailed distribution mentioned above, we observe another phenomenon that can be commonly found in both cross-attention and self-attention matrices.

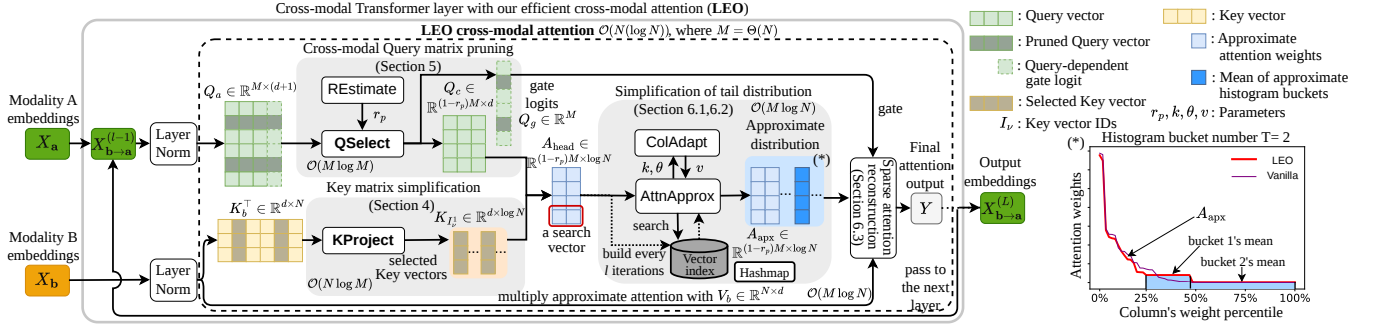


Figure 3: Overview of our proposed cross-modal attention LEO

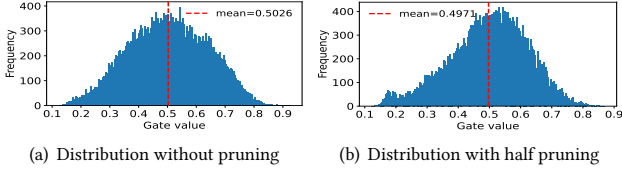


Figure 4: Gate scores distribution with head-specific element-wise gating [39]

**Definition 3.2. (Column-wise Attention Concentration)** Let  $A \in \mathbb{R}^{N \times N}$  denote an attention matrix, where each row  $A_{i,:}$  is a probability distribution returned by the softmax, i.e.,  $A_{ij} \geq 0$ ,  $\sum_{j=1}^N A_{ij} = 1$ .  $A$  exhibits *column-wise attention concentration* if there exists a small subset of column IDs  $I_v \subseteq \{1, \dots, N\}$  with  $k = |I_v| \ll N$ , when  $\mathbf{1}$  denotes the all-ones vector, such that

$$s_{I_v} = \frac{1}{N} \mathbf{1}^\top A_{:,I_v} \mathbf{1}. \quad (4)$$

In practice,  $s_{I_v}$  approaches 1 as  $N$  tends to be large. Column-wise attention concentration implies that most attention weights are assigned to a small subset  $I_v$ , which can be interpreted from an information-theoretic perspective as a low-entropy column distribution. When attention is uniformly distributed, the entropy reaches its maximum,  $\log N$ , with  $N$  columns. In contrast, as attention concentrates on a few columns, the entropy decreases, reaching zero in the extreme case where all weights lie on a single column. Empirically, training further reduces this entropy, indicating that only a small number of columns dominate the attention. This effect is illustrated in Figures 1(a) and 1(c), where attention weights progressively concentrate on a few columns during training epochs.

### 3.3 Token Importance in Cross-modal Attention

Previous sections suggest the sparsity in cross-modal interactions, indicating that many tokens carry weakly relevant information [6, 69]. Previous studies [2] and our preliminary experiments in Figure 4 show that pruning a large portion of tokens from one modality often causes only minor accuracy degradation. This observation implies that cross-modal representations contain redundant semantic information, where multiple tokens encode similar concepts.

To further analyze token importance across attention heads, we introduce an element-wise Query-dependent gating mechanism [39] to estimate each token’s contribution during cross-modal interaction. The gate representation is defined as

$$Y'_a = g(Y_a, X_a, W_\theta, \sigma) = Y_a \odot \sigma(X_a W_\theta),$$

where  $Y_a$  and  $X_a$  follow the same notation in Equation 1,  $W_\theta$  refers to the learnable gate parameters, and  $\sigma$  represents the sigmoid activation. The gate scores can be interpreted as an importance score that reflects how much each token contributes to the cross-modal interaction. Figure 4 shows the distribution of Query-dependent gate scores in cross-modal attention. In both cases, the scores follow an approximately unimodal Gaussian-like distribution centered around 0.5, with only a slight shift in the mean after pruning half of the Query vectors while preserving the overall shape of the distribution. This observation indicates that token importance is distributed continuously rather than dominated by a small set of informative tokens. As a result, a portion of tokens can be pruned with little impact on representation quality, since many tokens contribute only modestly to cross-modal aggregation.

### 3.4 Overview of LEO

To address efficient cross-modal attention, we propose LEO. As shown in Figure 3, LEO consists of four steps. First, a KProject (Algorithm 1) is leveraged to select columns of the transposed Key matrix for efficient attention computations. Second, we propose QSelect (Algorithm 2) to prune the Query vectors at a constant ratio  $r_p$  determined according to the modality importance in Section 5. Third, an attention approximation method called AttnApprox is designed to approximate the attention distribution with approximate histograms. As attention distributions vary with different inputs, we adaptively adjust the column subset size  $k$  based on our cost model during training. Finally, Algorithm 3 (detailed in Appendix A.3) integrates the algorithms above and obtains the final output of our cross-modal attention after our sparse attention reconstruction, which simplifies the multiplication between our approximate attention matrix and the Value matrix.

## 4 KEY MATRIX SIMPLIFICATION

In Section 3, we discuss the high column-wise attention concentration and long-tailed distribution in the cross-modal attention matrices. In Section 4.1, we exploit these characteristics to propose an efficient column subset selection method that runs in  $O(N \log M)$  time. In Section 4.2, we analyze our method to derive a theoretical error bound for large matrices.

#### 4.1 Column Subset Selection (CSS) of Key Matrix for Attention

The brighter (red) columns in Figure 1(c) can be selected to form an approximation matrix to multiply by matrix  $V$ , while the whole attention matrix is still calculated with time-consuming softmax. Considering the strict monotonically increasing property of softmax, it preserves the relative ordering of attention weights in each attention row, and thus we can apply the column subset selection (CSS) on the multiplication  $QK^\top$  to reconstruct the matrix while minimizing the CSS error  $E_{\text{Mul-CSS}}$  defined below.

**Definition 4.1. (CSS Error of Attention Logits)** Given two matrices  $Q \in \mathbb{R}^{M \times d}$ ,  $K \in \mathbb{R}^{d \times N}$  and a positive integer  $k$ , select  $k$  columns of  $QK^\top$  to form a matrix  $\tilde{C} \in \mathbb{R}^{M \times k}$  that minimizes the CSS error of attention logits:

$$E_{\text{Mul-CSS}} = \|QK^\top - \tilde{C}\tilde{C}^+(QK^\top)\|_F^2, \quad (5)$$

where  $\tilde{C}^+$  represents the Moore-Penrose inverse matrix of  $\tilde{C}$ , and  $\|\cdot\|_F^2$  denotes the square of Frobenius norm.

According to Definition 4.1, a classic method [35] applies a greedy local search to select  $k$  columns from  $QK^\top$  minimizing the reconstruction error. Starting from a random subset, it iteratively replaces columns with better candidates that reduce the CSS error, ensuring monotonic improvement until convergence to a local optimum. However, the explicit costly computation of  $QK^\top$  still requires a quadratic complexity in the selection process. Instead, we directly select the matrix  $K^\top$  to approximate  $QK^\top$  without computing the multiplication, as presented in KProject (Algorithm 1). In this case, we obtain the CSS error  $E_{\text{CSS-Mul}}$  below.

**Definition 4.2. (CSS Error of Key Submatrix)** Given two matrices  $Q \in \mathbb{R}^{M \times d}$ ,  $K \in \mathbb{R}^{d \times N}$  and a positive integer  $k$ , select  $k$  columns of  $K^\top$  that form a matrix  $C \in \mathbb{R}^{d \times k}$  that minimizes the CSS error of Key submatrix:

$$E_{\text{CSS-Mul}} = \|QK^\top - Q(CC^+K^\top)\|_F^2, \quad (6)$$

where  $C^+$  is the Moore-Penrose inverse matrix of  $C$ , and  $\|\cdot\|_F^2$  denotes the square of Frobenius norm.

As shown in Figure 5, we present the structure of our proposed KProject. Specifically, a set of rows  $I_\mu$  is first randomly sampled from the Query matrix  $Q$  (Line 1)<sup>1</sup>, and the corresponding submatrix  $Q_{I_\mu}$  is formed. The partial attention matrix  $Q_{I_\mu}K^\top$  is then computed, followed by a softmax to obtain the attention matrix  $A_{\text{sample}}$  (Lines 2). Next, a score vector  $s_{\text{col}}$  is formed by computing the columns' sums of the attention weights, which indicate the overall importance of each key with respect to the sampled Query submatrix (Line 3). The column IDs  $I_\nu$  of the top- $k$  largest entries in  $s_{\text{col}}$  are selected (Line 4), corresponding to the  $k$  most important columns of  $K^\top$ . Finally, the algorithm returns the sampled Query IDs  $I_\mu$ , the selected IDs  $I_\nu$ , and the attention matrix  $A_{\text{sample}}$  computed on the sampled Query submatrix (Line 5).

**Time complexity of KProject.** Let  $Q \in \mathbb{R}^{M \times d}$  and  $K \in \mathbb{R}^{N \times d}$ . Under condition  $k = O(\log N)$  (cf. Equation 9), the overall time complexity of KProject is  $O(N \log M)$ .

<sup>1</sup>According to [4, 13],  $\Theta(\log N)$  rows are sufficient to reconstruct an  $N \times N$  matrix with error bounds.

---

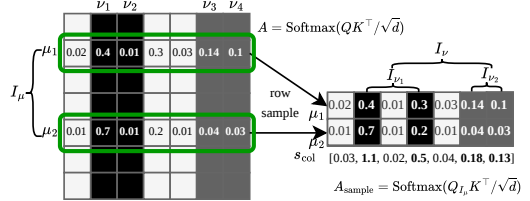
#### Algorithm 1: KProject( $Q, K, k$ ): Column subset selection for Key Matrix

---

**Input:** Query matrix  $Q$ , Key matrix  $K$ , selected column number  $k$   
**Output:** sampled Query IDs  $I_\mu$  on  $Q$ , selected Key IDs  $I_\nu$  after CSS on  $K^\top$ , attention matrix  $A_{\text{sample}}$  on sampled  $Q_{I_\mu}$

- 1 Random sample a list of rows  $I_\mu$  of  $Q$ ; // sample  $O(\log M)$  rows
- 2  $A_{\text{sample}} \leftarrow \text{softmax}(Q_{I_\mu}K^\top/\sqrt{d})$ ; //  $O(N \log M) E_{\text{Mul-CSS}}$  error
- 3  $s_{\text{col}} \leftarrow \mathbf{1}_{|I_\mu|}^\top A_{\text{sample}}$ ; // Get the column-wise sum of  $A_{\text{sample}}$
- 4  $I_\nu \leftarrow \text{argTopk}(s, k)$ ;
- 5 **return** ( $I_\mu, I_\nu, A_{\text{sample}}$ )

---



**Figure 5: An example of KProject.** Darker blocks indicate larger attention weights. We set  $k = 4$  and sample two query rows to compute a partial attention matrix, from which column scores are aggregated. The top- $k$  columns are then selected based on these scores (e.g., columns 2, 3, 6, and 7), illustrating how CSS identifies the most important tokens.

#### 4.2 Error analysis of Our CSS Method

Our proposed fast CSS method has a theoretical error, namely  $E_{\text{CSS-Mul}}$ , bounded by the CSS error  $E_{\text{Mul-CSS}}$ , which is presented in Theorem 4.3.

**THEOREM 4.3 (BOUNDED ERROR OF  $E_{\text{CSS-Mul}}$  BY  $E_{\text{Mul-CSS}}$ ).** Let  $Q \in \mathbb{R}^{M \times d}$  and  $K^\top \in \mathbb{R}^{d \times N}$ , and let  $K^\top = U\Sigma V^\top$  be the singular value decomposition of  $K^\top$ , where  $U = [u_1, \dots, u_r]$  and  $r = \text{rank}(K)$ . Assume that  $Q^\top Q$  is diagonal in the basis  $\{u_i\}_{i=1}^r$ , i.e.,  $u_i^\top Q^\top Q u_j = 0$ , for  $i \neq j$ , and define  $\alpha := \min_{i>k} \|Qu_i\|_2 > 0$ . Then

$$E_{\text{CSS-Mul}} \leq \frac{\|Q\|_2^2}{\alpha^2} (1 + \epsilon) E_{\text{Mul-CSS}}. \quad (7)$$

The proof of Theorem 4.3 is presented in Appendix A.5. Theorem 4.3 presents the relationship between the two errors and points out the influencing factors, including the approximation ratio of our method and the scaling behavior of matrix  $Q$ . In particular, if the ratio  $\|Q\|_2/\alpha$  of matrix  $Q$  along the unselected principal directions is close to 1, the errors of the two methods will not differ significantly. According to the proof in [13], we can easily provide the reconstruction error bound of  $E_{\text{Mul-CSS}}$  with randomized leverage score sampling: for some  $\epsilon_1 \in (0, 1)$ ,

$$E_{\text{Mul-CSS}} = \|QK^\top - \tilde{C}\tilde{C}^+QK^\top\|_F^2 \leq (1 + \epsilon_1) \|QK^\top - (QK^\top)_k\|_F^2 \quad (8)$$

when the sample number  $c$  satisfies  $c = (O(k \log k/\epsilon_1^2))$ . Therefore, combining with Theorem 4.3, our  $E_{\text{CSS-Mul}}$  can finally be bounded with

$$E_{\text{CSS-Mul}} \leq \frac{\|Q\|_2^2}{\alpha^2} (1 + \epsilon)(1 + \epsilon_1) \|QK^\top - (QK^\top)_k\|_F^2 \quad (9)$$

where the parameters are the same as in the previous theorems, after we sample at least  $c = \Theta(k \log k/\epsilon_1^2)$  columns [13].

**Discussion on column subset size.** In practice, we typically set at least  $k = O(\log N)$  and  $c$  reduces to  $O(\log N)$  since  $\log \log N$

---

**Algorithm 2:** QSelect( $Q, K, r_p$ ): Query Matrix Pruning

---

**Input:** Query matrix  $Q \in \mathbb{R}^{M \times (d+1)}$  from modality  $A$ , Key matrix  $K$  from modality  $B$ , Query matrix prune ratio  $r_p$  from REstimate (Appendix A.3)

**Output:** Query content matrix  $Q_c$  after pruning, remaining Query vector IDs  $I_g$ , gate scores  $g$

```
1  $Q_c \leftarrow Q[:, :d-1]$ ,  $Q_g \leftarrow Q[:, d]$ ; // split content & gate
2  $g \leftarrow \sigma_{\text{sigmoid}}(Q_g)$ ; // head-specific elementwise gate
3  $I_g \leftarrow \text{argTopk}(g, M - \lfloor r_p M \rfloor)$ ;
4  $Q_c \leftarrow \text{Gather}(Q_c, I_g)$ ;
5 return ( $Q_c, I_g, g$ )
```

---

can be treated as a constant for all practical input sizes. Notably, there is a trade-off between the error  $E_{\text{CSS-Mul}}$  and the time cost of Algorithm 1 determined by the selected column subset size, which means that a larger size can bring less CSS error but cost more time to select the columns. The details of determining the column subset size will be presented in Section 6.

## 5 CROSS-MODAL QUERY MATRIX PRUNING

In this section, we introduce a lightweight gating mechanism on the Query matrix to guide the selection of informative tokens (*i.e.*, pruning non-informative tokens) while preserving the overall  $O(Md)$  scale of the Query matrix. This aligns with the semantics of cross-modal attention, where modality  $B$  provides information to modality  $A$ . In addition, we highlight our mechanism for determining the Query matrix prune ratio based on estimated modality importance with theoretical bounds.

### 5.1 Gate-based Query Matrix Pruning

We consider cross-modal attention in which the Query matrix  $Q_a$  is derived from modality  $A$ , while the Key and Value matrices  $K_b, V_b$  come from modality  $B$ . We introduce the attention with a Query matrix pruning mechanism, namely QSelect (Algorithm 2), where a learned gate  $g$  models the importance of each Query vector, and a top- $k$  selection for  $g$  is applied to retain Query vector IDs  $I_g$  according to the prune ratio  $r_p$ . Pruned Query vectors are skipped in the attention computation, and their outputs are replaced by the mean Value vector  $\bar{V}$  in the final attention output.

Specifically, the input Query matrix  $Q$  is split first into a content matrix  $Q_c$  for attention computation and gate logits  $Q_g$  (Line 1). The gate scores  $g$  are then obtained by applying a sigmoid function to  $Q_g$ , producing a head-specific elementwise gating result (Line 2). Then, the IDs  $I_g$  of the top  $M - \lfloor r_p M \rfloor$  Query vectors, sorted by gate scores, are selected (Line 3), and only the corresponding  $Q_c$  are collected to form the pruned Query matrix (Line 4).

*Example 5.1.* Given Query matrix  $Q = [q_1, q_2, q_3, q_4]^T$  and  $r_p = 0.5$ . Assume we have  $Q[:, d] = [q_{1d}, q_{2d}, q_{3d}, q_{4d}]^T = [0.5, -2, 1, -1.5]^T$ . Then,  $g = [0.6, 0.1, 0.7, 0.2]^T$  and  $I_g = \{1, 3\}$ . Thus, after pruning,  $Q_c = [q_1, q_3]^T$ .

**Time Complexity of QSelect.** Given the input Query matrix  $Q \in \mathbb{R}^{M \times d}$ , Key and Value matrix  $K, V \in \mathbb{R}^{N \times d}$ . Therefore, the overall time complexity of QSelect is  $O(M \log M)$  when  $d \ll M$ .

**Analysis of Query matrix pruning.** QSelect (Algorithm 2) can be considered as an estimator of cross-modal attention with an error

bound. Specifically, only informative Query vectors will undergo attention computation, while the other ones are approximated by the mean Value  $\bar{V}$ , which acts as a global cross-modal prior and will be discussed further in Section 6.3. Intuitively, when a Query vector contributes little to attention weights, its attention distribution is small and relatively uniform, and its error between the exact attention output is also small. Therefore, the proposed approximation remains a first-order approximation of vanilla attention. More analysis will be elaborated in Section 6.3.

**THEOREM 5.2 (ERROR OF QUERY MATRIX PRUNING).** *Given the  $i$ -th gate score  $g_i$  of learned gate  $g$ , Value matrix  $V$ , its column mean  $\bar{V}$ , the  $i$ -th row of the attention weight matrix  $A_i$  with vanilla attention, the error  $E_{\text{project}}$  of the attention output in the  $i$ -th row due to QSelect satisfies the following:*

$$E_{\text{project}} = \|g_i \cdot A_i \cdot V - g_i \cdot \bar{V}\|_F \leq 2g_i \|V\|_F. \quad (10)$$

The proof of Theorem 5.2 is presented in Appendix A.6. Theorem 5.2 shows that the error  $E_{\text{project}}$  is small enough to estimate attention when the gate score  $g_i$  approaches zero. When the cross-modal attention contribution of Key/Value matrix modalities increases, the information carried by the Query matrix increases proportionally, suggesting that more Query vectors must be retained to keep the approximation error under control.

### 5.2 Query Matrix Prune Ratio Estimation

When the output is primarily driven by attention computed from Query vectors, aggressively pruning the Query matrix is erroneous, and we retain more Query vectors. Conversely, when the output is mainly explained by the Value-mean term, a larger Query matrix prune ratio leads to efficient computation and introduces negligible error. Hence, this subsection estimates the Query matrix prune ratio across different attention inputs based on the relative importance of the input modality contributions.

Due to space restrictions, the lightweight Query matrix prune ratio estimator, namely REstimate( $Q, K, Y$ ), is provided in Appendix A.3. In a nutshell, the estimated prune ratio  $r_p$  computed in the last iteration is kept as the input to the current computation and outputs the ratio for QSelect. Specifically, during training, we first compute the representations  $a$  of modality  $A$ ,  $b$  of modality  $B$ , and  $y$  from the Query matrix  $Q$ , Key matrix  $K$ , and attention output  $Y$  from the last iteration, respectively. Then, the cosine similarity between each modality and the attention output is used to measure the alignment of the modalities with the output. Based on the relative proportion  $S_A$  of similarity to  $A$ , defined as  $|a^T y| / (|a^T y| + |b^T y|)$ , the prune ratio  $r_p$  is updated as  $(1 - S_A)$ .

**Analysis of the Query matrix prune ratio.** We provide a theoretical analysis showing that, in addition to measuring angular alignment, the cosine-ratio score can also be leveraged to indicate the relative contribution of the two modalities, indicating that it serves as a reliable signal for controlling the pruning ratio. Here, we adopt mean-pooling normalized results as representations, and other representations are also allowed. For a fixed sample, denote by  $\tilde{y} \in \mathbb{R}^d$ , the representation of the cross-modal attention output. Let  $a, b \in \mathbb{R}^d$  be the representations of modalities  $A$  and  $B$ , respectively. Then, we can define the estimator

$$\hat{r}_A := |a^T \tilde{y}| / (|a^T \tilde{y}| + |b^T \tilde{y}|). \quad (11)$$

**THEOREM 5.3 (CONCENTRATION OF BATCH-WISE ESTIMATOR).** Assume the estimator uses the batch-wise  $S_A := \frac{1}{B} \sum_{i=1}^B \hat{p}_A(i)$ . Each  $\hat{p}_A(i) \in [0, 1]$  denotes i.i.d. samples. Then, for any  $\delta \in (0, 1)$ , with probability at least  $1 - \delta$ ,

$$|S_A - \mathbb{E}[\hat{p}_A]| \leq \sqrt{\ln(2/\delta)/2B}, \quad (12)$$

where  $B$  is the batch size and  $S_A$  is the relative proportion of cosine-similarity to modality A.

The proof of Theorem 5.3 is presented in Appendix A.10. Under our representations, the cosine-ratio estimator approximates this contribution ratio with a bounded error. Moreover, since the estimator is bounded in  $(0, 1)$ , the batch-wise estimator concentrates on its expectation at an  $O(1/\sqrt{B})$  rate by Hoeffding’s inequality. Therefore, Theorem 5.3 confirms that the proposed batch-wise similarity-based estimator provides an estimate of the true cross-modal contribution ratio. This demonstrates that adaptively allocating Query vectors in proportion to our proposed ratio has a theoretical foundation to minimize approximation error in multi-modal transformers.

## 6 SIMPLIFICATION OF TAIL DISTRIBUTION OF ATTENTION

The sparsity in the attention tokens indicates that many tokens carry small weights. However, the attention distributional tail cannot be completely ignored. We propose to employ histograms to approximate the attention weight distribution with theoretical guarantees on KL divergence and design a cost model to dynamically adjust the parameters in LEO.

### 6.1 Histogram for Attention Approximation

After selecting the top- $k$  columns to capture the head distribution in  $A_{\text{head}}$ , we approximate its tail entries to obtain the full attention output. We partition all the Key columns into a *head subset*  $I_v^1$  and  $T$  *tail subsets*  $\{I_v^t\}_{t=2}^{T+1}$ . The head subset computes attention  $A_{\text{head}} = \text{softmax}(Q(K^\top)_{I_v^1}/\sqrt{d})$ , while the tail subsets are used for approximation by histograms [20].

For each row  $a_j$  in  $A_{\text{head}}$ , we sort this head vector to compute the variance contribution ratio using squared  $\ell_2$  norms [30]. We define the *variance contribution ratio*  $\vartheta_j$  as follows: for each search vector  $\tilde{a}_j = \text{sort}(A_{\text{head}}[j, :], \text{desc})$ ,

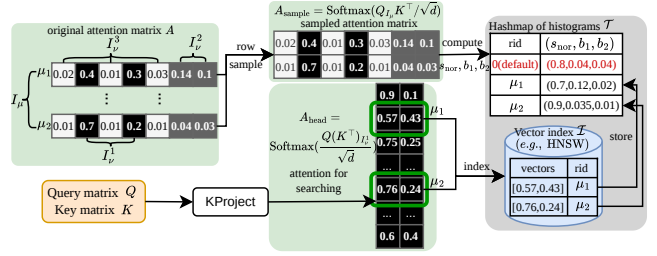
$$\vartheta_j = \tilde{a}_j[h|I_v^1| : ] / \tilde{a}_j[: h|I_v^1|], \quad (13)$$

where  $h$  is a hyperparameter that controls the partition of search vectors. Those vectors are filtered by  $\vartheta_j$  as follows: (i) those with  $\vartheta_j$  exceeding a threshold  $\theta$  (to be determined in the next subsection) are considered *skewed*, whereas (ii) others are considered *flat*.

(i) For skewed search vectors, we retrieve their corresponding histogram via ANN search on sorted head vectors. In particular, for each sampled row ID  $i \in I_\mu$ , we first build the *vector index*  $\mathcal{I}$  (e.g., HNSW [31]) on  $A_{\text{head}}[I_\mu, :]$  and store normalized score of the head vector and a histogram of attention means in a *hashmap*  $\mathcal{T}$ : for  $t \in \{1, \dots, T\}$ ,

$$s_{\text{nor}}^i = \sum A_{\text{sample}}[i, I_v^1], \text{ and } b_t^i = \text{mean}(A_{\text{sample}}[i, I_v^{t+1}]), \quad (14)$$

where  $s_{\text{nor}}^i$  captures the total probability mass in the head, while  $b_1^i, \dots, b_T^i$  are the average masses in each (tail) bin. The details



**Figure 6: Illustration of building the histogram index with  $|I_v^1| = |I_v^2| = 2$ ,  $T = 2$ . After obtaining  $I_v^1$  and  $I_v^2$ , we compute  $A_{\text{pre}}$ . The sorted rows  $\text{sort}(A_{\text{pre}}[\mu_1, :]) = [0.57, 0.43]$  and  $\text{sort}(A_{\text{pre}}[\mu_2, :]) = [0.76, 0.24]$  are inserted into vector index. The hashmap  $\mathcal{T}$  stores the corresponding histograms:  $(s_{\text{nor}}^1, b_1^1, b_2^1) = [0.7, 0.12, 0.02]$  and  $(s_{\text{nor}}^2, b_1^2, b_2^2) = [0.9, 0.035, 0.01]$ , with a default histogram  $(s_{\text{nor}}^0, b_1^0, b_2^0) = [0.8, 0.04, 0.04]$  for flat vectors.**

(IndexBuild) are straightforward but verbose and, hence, presented in Appendix A.3. It constructs the aligned structures  $\mathcal{I}$  and  $\mathcal{T}$  with time complexity of  $O(N \log M)$ . To balance construction cost and search efficiency,  $\mathcal{I}$  and  $\mathcal{T}$  are built every  $l$  iterations. Figure 6 illustrates this process with a concrete example. We perform an ANN search to retrieve normalized score and histogram bucket means for the skewed vectors from  $\mathcal{T}$  to obtain the approximate attention  $A_{\text{app}}$  via row normalization. Figure 7 shows how we use the skewed vectors to search the index and histogram to approximate attention.

(ii) A flat vector  $a_j$  is approximated by a default histogram, where  $s_{\text{nor}}^0 = \text{mean}(s_{\text{nor}}^{i \geq 1})$  and  $b_t^0 = (1 - s_{\text{nor}}^0)/(N - |I_v^1|)$ .

The overall pseudo-code,  $\text{AttnApprox}(A_{\text{head}}, \mathcal{I}, \mathcal{T}, h, \theta)$ , is elaborated in Appendix A.3. Its key ideas are that by combining the search results for skewed and flat vectors, we obtain  $s_{\text{nor}}$  and  $(b_1, \dots, b_T)$ . The approximate attention is then computed by  $A_{\text{app}} = s_{\text{nor}} \odot A_{\text{head}}$ , with histogram bucket means  $(b_1, \dots, b_T)$ . An example is shown in the RHS of Figure 7.

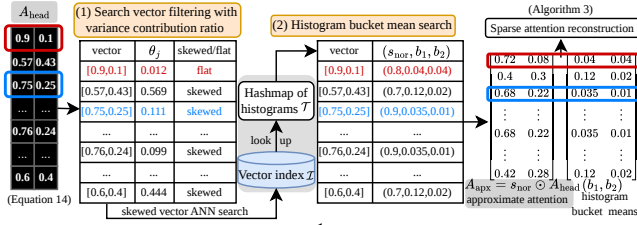
**Remarks.** As shown in Figure 19 in the Appendix, increasing bucket number  $T$  beyond 2 brings diminishing returns; therefore, we set  $T = 2$  in practice. Conversely, ignoring the attention distributional tail leads to a much larger KL divergence, and Value vectors corresponding to the tail cannot participate in the attention.

**Analysis of distribution approximation.** We use KL divergence, a distribution-based metric, to evaluate our attention approximation, as KL divergence treats attention weights as probability distributions and penalizes mass<sup>2</sup> assigned to less relevant regions. We present the analysis result in Theorem 6.1.

**THEOREM 6.1 (KL DIVERGENCE EXPECTATION).** Consider one row distribution of an attention matrix after softmax, where each  $p_i = \frac{e^{b_i}}{Z}$ ,  $Z = \sum_{j=1}^N e^{b_j}$ , sorted in decreasing order of attention weight. Denote  $\hat{p}_i$  by our distribution approximation of the same row. Assume attention weights in one row follow a truncated power law  $p_{(i)} = \frac{i^{-\alpha}}{H_{N,\alpha}}$ ,  $\alpha > 1$ , where  $H_{N,\alpha} = \sum_{j=1}^N j^{-\alpha}$ . The KL divergence between  $p_i$  and  $\hat{p}_i$  satisfies the following:

$$\mathbb{E}[\text{KL}(p \parallel \hat{p})] = \Theta(\log N \cdot k^{1-\alpha}). \quad (15)$$

<sup>2</sup>Mass refers to the total probability assigned to a subset of elements.



**Figure 7: Illustration of with  $|I_v| = 2, T = 2, h = 0.5$ . In phase (1), all head vectors are filtered by variance contribution ratio; the flat vector  $[0.9, 0.1]$  falls below the threshold  $\theta = 0.03$ . In phase (2), skewed search vector  $[0.75, 0.25]$  retrieves the nearest neighbor  $[0.76, 0.24]$  from vector index and obtains  $(0.9, 0.035, 0.01)$  from hashmap  $\mathcal{T}$ . The approximate attention is  $0.9 \times [0.75, 0.25] = [0.68, 0.22]$ , with histogram bucket means  $(0.035, 0.01)$  for subsequent computation.**

Due to page limitations, we present the detailed proof in Appendix A.7. When  $k = O(\log N)$ , Equation 15 gives  $\mathbb{E}[\text{KL}(p\|\hat{p})] = \Theta((\log N)^{2-\alpha})$ , which approaches zero when  $\alpha$  is large. This indicates that sharper attention distributions yield smaller approximation errors.

## 6.2 Cost Model for Parameter Setting

Fixed parameter values in LEO, *i.e.*,  $k$  and  $\theta$  cannot adapt to the evolving and gradually skewed attention distributions across epochs. In this subsection, we present the cost model for setting  $k$  and  $\theta$ . We first briefly present the rationale for the cost model. Specifically, we present Theorem 6.2 to compare the KL divergence brought by histogram bucket mean search between attention rows with skewed search vectors and those with flat vectors.

**THEOREM 6.2 (EXPECTATION OF KL DIVERGENCE COMPARISON).** Denote  $\text{KL}_{\text{mass}}$  to be the KL divergence that comes from mismatching the total probability mass assigned to each column subset, which is independent of how probabilities are distributed inside the subsets. Let  $\text{KL}_{\text{mass}}^{\text{skewed}}$  and  $\text{KL}_{\text{mass}}^{\text{flat}}$  denote  $\text{KL}_{\text{mass}}$  over attention rows with skewed and flat search vectors in the same  $A_{\text{head}}$ , respectively. Under the same second-order approximation [41], the following holds

$$\mathbb{E}[\text{KL}_{\text{mass}}^{\text{skewed}}] \leq \mathbb{E}[\text{KL}_{\text{mass}}^{\text{flat}}]. \quad (16)$$

Skewed vectors have a stronger dependency on the Query matrix in the allocation of probability mass across column subsets, which reduces the residual variance governing  $\text{KL}_{\text{mass}}$  across attention as presented in our proof in Appendix A.8, and further leads to a smaller  $\mathbb{E}[\text{KL}_{\text{mass}}^{\text{skewed}}]$ . Given the true skewed vector ratio  $v$  measuring the ratio of skewed vector number to the total search vector number, the total expectation of the KL divergence from probability mass in the whole attention matrix can be written as

$$\mathbb{E}[\text{KL}_{\text{mass}}] = v \cdot \mathbb{E}[\text{KL}_{\text{mass}}^{\text{skewed}}] + (1 - v) \cdot \mathbb{E}[\text{KL}_{\text{mass}}^{\text{flat}}] \quad (17)$$

When  $v$  is large, filling a global mean in each row fails to adapt to individual queries, and the resulting large KL gap is mainly determined by how the probability mass assigned to each column subset varies across attention rows.

**Cost model.** Since the skewed vector ratio  $v$  only influences the total search time of our index, we propose a time cost model to balance the search time of each vector and the total vector number. When employing HNSW, the search time complexity can be denoted

as  $O(N_v k \log N_d \cdot \text{efs})$  with vector number  $N_v$ , max degree in HNSW  $N_d$ , size of candidate set  $\text{efs}$ , and the dimension of each vector  $k$ . Accordingly, the search cost can be written as

$$C_{\text{search}} = \text{efs} \cdot k \log N_d \cdot N_v \quad (18)$$

where large  $k$  and  $N_v$  means more time cost. In practice, we maintain  $\text{efs} \cdot \log N_d$  and change  $k, N_v$  to explore efficient settings. The parameters  $k$  and  $\theta$  jointly control the efficiency-accuracy trade-off as follows: Increasing  $k$  improves approximation quality but increases per-search cost, while increasing  $\theta$  reduces the number of skewed vectors that require costly search. Thus,  $k$  determines the cost per search and  $\theta$  determines the number of skewed vectors, allowing them to compensate for each other in practice.

**Remarks.** While the verbose pseudo-code of attention approximation AttnApprox is presented in Appendix A.3, we highlight the key considerations here. First, we introduce a subroutine ColAdapt to adaptively control the size  $k$ , and the threshold  $\theta$  based on the error  $\epsilon$  between the true skewed vector ratio  $v$  and the target ratio  $\tau$ , to balance computational cost and attention quality, as the training progress. Second, unlike updating  $k$  proportionally to the error, we perform an exponentiated update on  $\theta$  since it operates on a positive scale and benefits from relative adjustments, which ensures smooth changes in the case of large errors. Finally, ColAdapt returns  $(k, \theta)$  for the current computation in AttnApprox.

## 6.3 LEO Cross-modal Attention Mechanism

Based on the previously introduced components, we put them together for our LEO attention mechanism.

**Overall mechanism.** We present our LEO attention mechanism for a single iteration in Algorithm 3. In the first iteration, we initialize parameters  $r_p, k, \theta$ , respectively. Then, these parameters are adaptively adjusted as presented in Section 6.2. Lines 3-7 compute the histogram's bin widths, and Line 10 divides  $I_v$  by the bin widths. Lines 8-11 use QSelect and KProject to compute  $A_{\text{head}}$ . Lines 12-13 maintain a lightweight index for vector search that contains a hashmap to vectors' histograms. In Line 14, AttnApprox approximates the attention  $A_{\text{head}}$ . The internal result  $U$  obtained in Lines 16 is computed with the reduced vectors in the approximate histograms and the Value matrix. Lines 17-20 fill the positions of the Query vectors pruned by QSelect in Line 8 with the mean from the Value matrix of modality  $B$ , while others keep the result in  $U$ . Finally, the outputs are used as input for the next iteration of LEO.

Overall, as highlighted in Lines 11 and 16, LEO improves computational efficiency by reducing both Query-Key and attention-Value multiplications, while providing corresponding error bounds for each approximation. It should be remarked that the vector search index  $\mathcal{I}$  and hashmap of histograms  $\mathcal{T}$  are rebuilt every  $l$  iterations to balance construction cost and search efficiency.

**Analysis on sparse attention reconstruction.** We analyze the consistency between our contribution ratio of modality  $A$  and the estimated ratio of Equation 11 in Section 5.2. Specifically,  $A_0$  in Line 20 of Algorithm 3 admits an algebraic decomposition

$$\tilde{y} = \tilde{y}^{(B \rightarrow A)} + \tilde{y}^{(B)}. \quad (19)$$

$\tilde{y}^{(B)}$  is a Value-mean baseline depending only on modality  $B$ , zeroed at unpruned Query vector positions, whereas  $\tilde{y}^{(B \rightarrow A)}$  is a Query-dependent term for modality  $A$ , zeroed at pruned positions. Then,

---

**Algorithm 3: LEO cross-modal attention**


---

**Input:** Input Query matrix  $Q \in \mathbb{R}^{M \times (d+1)}$ , Key matrix  $K \in \mathbb{R}^{N \times d}$ , Value matrix  $V$ , number of approximate histogram buckets  $T$ , vector split ratio  $h$ , target ratio  $\tau$ , vector index  $I$  and hashmap of histograms  $\mathcal{T}$  for the current iteration

**Output:** Final attention output  $Y$ , parameters  $(k, v, \theta)$ , vector index  $I$ , hashmap of histograms  $\mathcal{T}$  for next iteration

```

1  $r_p \leftarrow \text{REstimate}(Q, K, Y);$  // Compute  $Q$  prune ratio in Appendix A.3
2  $(k, \theta) \leftarrow \text{ColAdapt}(k, v, \theta, \tau);$  // Compute column subset size, variance contribution threshold in Appendix A.3
3  $W \leftarrow \emptyset;$  // Initialize histogram bin width set  $W$ 
4  $k_1 \leftarrow k, k_{\text{all}} \leftarrow k_1, W.\text{add}(k_1);$ 
5 for  $t \leftarrow 2$  to  $T$  do
6    $k_t \leftarrow k^{t-1};$  // Compute histogram bin width
7    $k_{\text{all}} \leftarrow k_{\text{all}} + k_t, W.\text{add}(k_t);$ 
8  $(Q_c, I_g, g) \leftarrow \text{QSelect}(Q, r_p);$  // Algo. 2 in Sec. 5
9  $(I_\mu, I_v, A_{\text{sample}}) \leftarrow \text{KProject}(Q_c, K, k_{\text{all}});$  // Algo.1 Sec. 4
10  $\{I_v^t\}_{t=1}^T \leftarrow \text{Divide}(I_v, W), I_v^{T+1} \leftarrow N \setminus I_v;$ 
11  $A_{\text{head}} \leftarrow \text{Softmax}(Q(K^\top)_{I_v^1} / \sqrt{d});$  //  $O(M \log N)$ 
12 if update the index every  $l$  iterations then
13    $(I, \mathcal{T}) \leftarrow \text{IndexBuild}(A_{\text{sample}}, A_{\text{head}}, I_\mu, \{I_v^t\}_{t=1}^{T+1});$ 
    //  $O(N \log M)$  in Appendix A.3
14  $(A_{\text{apx}}, (b_1, \dots, b_T), v) \leftarrow \text{AttnApprox}(A_{\text{head}}, I, \mathcal{T}, h, \theta);$ 
15 // Sparse attention reconstruction
16  $U \leftarrow A_{\text{apx}} V[I_v^1, :] + \sum_{t=1}^T (b_t \sum_{i \in I_v^{t+1}} V[i, :]);$  //  $O(M \log N)$ 
17  $\bar{V} \leftarrow \frac{1}{M} \mathbf{1}^\top V;$  // Compute the column-wise mean of  $V$ 
18  $A_0 \leftarrow \text{Replicate}(\bar{V}, M);$  // Replicate  $M$  rows using  $\bar{V}$ 
19 for  $i \leftarrow 1$  to  $|I_g|$  do
20    $A_0[I_g[i], :] \leftarrow U[i, :];$  // use  $U$  to reconstruct  $A_0$ 
21  $Y \leftarrow A_0 \odot g;$  // Gate the attention output
22 return  $(Y, ((k, v, \theta), (I, \mathcal{T})))$ 

```

---

we can define our contribution ratio of modality  $A$  as

$$\rho_A := \frac{\|\tilde{y}^{(B \rightarrow A)}\|_2}{\|\tilde{y}^{(B \rightarrow A)}\|_2 + \|\tilde{y}^{(B)}\|_2} \in [0, 1]. \quad (20)$$

Since the pruning operation only affects the term  $\tilde{y}^{(B \rightarrow A)}$  while leaving the term  $\tilde{y}^{(B)}$  unchanged, the sensitivity of the attention output to pruning is controlled by their relative  $\ell_2$  norms.

Theorem 6.3 bounds the estimation error between  $\rho_A$  in Equation 11 and  $\hat{\rho}_A$  in Equation 20, showing that the cosine-ratio score captures the relative contribution of the Query-dependent term versus the Value-mean baseline term. The error is maximized at  $\rho_A = 0.5$  and decreases otherwise, implying that a larger modality contribution imbalance yields a more accurate estimation, thereby supporting our  $A_0$  reconstruction in Algorithm 3. The detailed proof is presented in Appendix A.9.

**THEOREM 6.3 (BOUNDED ERROR OF CONTRIBUTION RATIO ESTIMATION).** *Let  $\tilde{y} = \tilde{y}^{(B)} + \tilde{y}^{(B \rightarrow A)}$  be the exact decomposition in Eq. (19), and define  $\rho_A$  and  $\hat{\rho}_A$  by Eq. (20) and Eq. (11), respectively. Then, the estimator  $\hat{\rho}_A$  satisfies the bound*

$$|\hat{\rho}_A - \rho_A| \leq \frac{\alpha_A(1 - \rho_A) + \beta_B \rho_A}{\beta_A \rho_A + \alpha_B(1 - \rho_A)}. \quad (21)$$

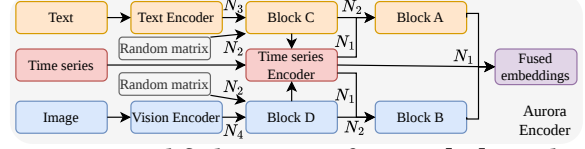


Figure 8: Simplified structure of Aurora [54] encoder.

Table 2: Statistics of the datasets used

| Dataset                | Domain             | Frequency | Length | Prediction lengths | Training samples |
|------------------------|--------------------|-----------|--------|--------------------|------------------|
| Health ( $H_i$ )       | Influenza Patients | Weekly    | 1,389  | 12/24/36/48        | 76/64/52/40      |
| Energy ( $Ey_i$ )      | Gasoline Prices    | Weekly    | 1,479  | 12/24/36/48        | 148/136/124/112  |
| Weather ( $W_i$ )      | Daily Temperature  | Daily     | 3,621  | 48/96/192/336      | 802/754/658/514  |
| Environment ( $Et_i$ ) | Air Quality Index  | Daily     | 11,102 | 48/96/192/336      | 642/594/498/354  |

where  $\alpha_A, \alpha_B \in [0, 1]$  quantify the alignment with the Value-mean baseline term, and  $\beta_A, \beta_B \in [0, 1]$  quantify the alignment with the cross-modal attention term.

**Overall Time Complexity.** The overall time complexity is  $O(M \cdot \log MN + N \log M)$ . In practice, assuming  $M = \Theta(N)$ , the final time complexity is written as  $\Theta(N \log N)$ .

## 7 EXPERIMENTAL EVALUATION

In this section, we empirically evaluate LEO within the Aurora framework [54], a strong baseline foundation model that employs the vanilla cross-modal attention mechanism. Our source code and appendix are publicly available.<sup>3</sup>

### 7.1 Experimental Setup

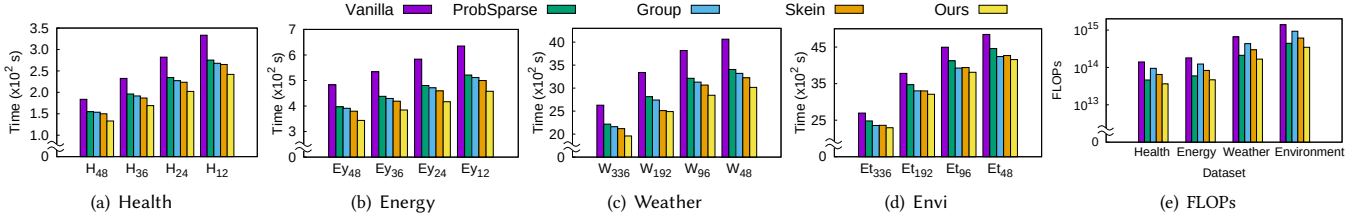
**Environment.** All experiments were conducted on a server configured with a single NVIDIA A800-80GB-PCIe GPU and a 14 vCPU Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz. Our software stack was built on Python 3.10, with PyTorch version 1.10.0 serving as the primary deep learning framework.

**Task and metrics.** We evaluate LEO with multimodal time-series forecasting, where cross-modal attention is the key for modality integration and alignment [54, 72]. By replacing only the cross-modal attention module in an existing framework, we enable a controlled comparison where performance differences primarily reflect those of various attention mechanisms. Efficiency is measured by total training time/runtime and attention FLOPs, while effectiveness is evaluated using MSE, MAE, and RSE (lower is better).

**Datasets.** We evaluate LEO on multimodal datasets from the Time-MMD Time Series Archive [29] and TimeText Corpus [23]. We use the four datasets with the longest time series, namely Health, Energy, Weather, and Envi. Table 2 presents the statistics of the datasets, where  $H_i$  denotes Health with a prediction length of  $i$ .

**Implementation details.** The Aurora framework contains an encoder with four cross-modal attention blocks interacting among text, vision, and time series modalities, namely blocks A, B, C, and D for convenience, each with different input lengths. Figure 8 shows the Aurora encoder structure [54]. Here,  $N_1$  is the Query length for blocks A and B;  $N_3$  and  $N_4$  are the Key/Value lengths of blocks C and D; and  $N_2$  is the Key/Value length of blocks A and B or the Query length of blocks C and D. For fair comparison in training time, all methods use identical hyperparameter settings, with a unified

<sup>3</sup><https://github.com/csrao/LEO.git>



**Figure 9: Efficiency evaluation on four datasets across all prediction lengths<sup>5</sup>. H, Ey, W, Et refer to Health, Energy, Weather, and Envi, respectively. Subscripts denote prediction lengths. (a–d) Training time. (e) FLOPs.**

selection budget (e.g., sampled or representative rows/columns) consistently controlled across attention variants. Datasets are split into 80%/10%/10% for training/validation/testing, and samples are generated via a sliding window. We evaluate under different prediction lengths<sup>4</sup> {12, 24, 36, 48} for Health and Energy, {48, 96, 192, 336} for Weather and Envi, which refers to the number of future time steps a model predicts ahead based on the observed input sequence. All hyperparameters are provided in Appendix A.17.

**Baselines.** To evaluate the performance of LEO, we benchmark four representative attention methods, including Vanilla and several sparse attentions with similar time complexity. All baselines are adapted to the cross-modal setting with two different input lengths, denoted as  $M$  and  $N$  for the Query and Key/Value matrices, respectively. We have included the following baselines: **Vanilla Attention** [49] (Vanilla): It is proposed in the original transformer architecture. **ProbSparse Attention** [73] (ProbSparse): It is a representative sparse attention operated on the Query matrix, reducing complexity to  $O(N \log N)$ . **Group Attention** [27] (Group): It employs k-means on the Key matrix and proposes group softmax to approximate softmax attention, achieving a complexity  $O(GN)$ , where  $G$  is the group number. **SkeinAttention** [5] (Skein): It performs an attention mechanism on sampled vectors in the Key matrix with a specific probability, yielding  $O(N \log N)$  complexity.

## 7.2 Overall Evaluation

We evaluate both the efficiency and prediction accuracy of the proposed attention mechanism.

**Efficiency.** Since a longer input length usually results in a lower MSE, we report the training time at a maximum input length  $2^{12}$  for all methods. As shown in Figure 9, LEO consistently achieves the lowest training time in four datasets in all prediction lengths. First, our method is significantly faster than Vanilla because it reduces the time complexity from  $O(N^2)$  to  $O(N \log N)$ . For example, when the prediction length is 48, LEO achieves speedups of 27.5%, 28.9%, 25.4%, and 14.9% compared to Vanilla in Health, Energy, Weather, and Envi, respectively. Second, compared to other efficient attentions, LEO remains consistently faster. For instance, in Health, LEO saves around 10% training time compared to the fastest baseline (Skein). This is because, although all efficient methods select "informative" tokens to compute attention, LEO facilitates a more efficient and effective selection of informative tokens when those tokens dominate overall attention. It should be remarked that on Envi, the batch size is reduced to avoid GPU memory overflow,

which increases I/O overhead and partially limits speedup. To further investigate efficiency, we compare FLOPs in the four datasets (Figure 9(e)). The result shows that LEO reduces FLOPs by up to  $3\times$  relative to Vanilla, which verifies that our method significantly reduces the number of Query-Key multiplications.

**Accuracy.** Table 3 reports the forecast accuracy of the compared attentions in four datasets with input length  $2^{12}$ . For most datasets, LEO achieves performance comparable to or better than existing baselines because LEO preserves tokens relevant to the prediction. Moreover, our method obtains better results in Weather and Envi since the number of training samples in these two datasets is much larger than that in Health and Energy, leading to better performance in fully supervised settings. It should be noted that Vanilla has the same average rank as other efficient cross-modal attentions, indicating the importance of preserving and amplifying key information in multimodal tasks.

## 7.3 Detailed Evaluation

**Efficiency scalability.** We measure the end-to-end training time in Envi with a prediction length 48 while varying the input lengths  $N_1$ ,  $N_2$ ,  $N_3$ , and  $N_4$  to explore the efficiency and scalability of LEO.

From Figure 10, we observe that the training time of all methods, when scaling  $N_1$  and  $N_4$ , increases rapidly as the input length increases. This result is consistent with the theoretical cost of cross-modal attention, in which the dominant computation arises from Query and Key interactions. In contrast, the time increases slowly with respect to  $N_3$ , indicating that these components participate in fewer cross-modal attention computations when training Aurora. Across all settings, our method consistently achieves the shortest runtime, which increases slowly with larger input length. The results empirically validate that the proposed token selection mechanism in Section 4 and Section 5 effectively reduces the interaction space of cross-modal attention, hence, improving scalability.

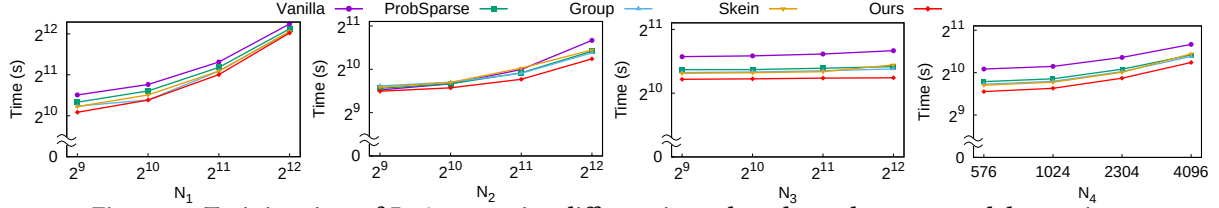
**Accuracy scalability.** Figure 11 reports the prediction MSE across different input lengths in Health with the prediction length 48. Here, we present the MSE scalability across two input lengths  $N_1$  and  $N_2$  for the time-series modality. Results on  $N_3$  and  $N_4$  are provided in Appendix A.13. Across all lengths, LEO achieves a performance comparable to or better than the other attention baselines and shows less prediction MSE when  $N_1$  becomes larger. When varying  $N_2$  from  $2^9$  to  $2^{12}$ , the MSE of our method exhibits an overall decreasing trend and achieves near-best or best performance under larger  $N_2$  settings because longer time series inputs provide more

<sup>4</sup>Larger prediction lengths indicate fewer training samples in one dataset due to the sliding-window strategy in time series forecasting.

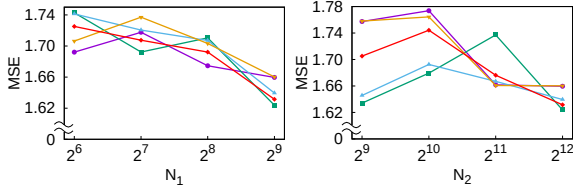
<sup>5</sup>As the prediction length increases, the runtime degrades for all methods because larger prediction lengths reduce the number of sliding window training samples.

**Table 3: Performance comparison of different cross-modal (CM) attention mechanisms across four datasets (Lower is better). The best and second-best results are highlighted in bold and underline, respectively.**

| Attention methods |     | CM Vanilla    |               |               | CM ProbSparse |               |               | CM Group      |               |               | CM Skein      |               |               | Ours          |               |               |
|-------------------|-----|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| Metrics           |     | MSE           | MAE           | RSE           | MSE           | MAE           | RSE           | MSE           | MAE           | RSE           | MSE           | MAE           | RSE           | MSE           | MAE           | RSE           |
| Health            | 12  | 1.5539        | 0.8951        | 1.0845        | 1.5923        | <u>0.8449</u> | 1.0978        | 1.6254        | 0.9657        | 1.1091        | <u>1.4684</u> | 0.8822        | <u>1.0542</u> | <b>1.4195</b> | <b>0.8297</b> | <b>1.0365</b> |
|                   | 24  | 1.8082        | 0.8436        | 1.1576        | <b>1.7674</b> | <b>0.8368</b> | <b>1.1445</b> | 1.8130        | 0.8440        | 1.1592        | 1.7970        | 0.8407        | 1.1540        | <u>1.7881</u> | <u>0.8389</u> | <u>1.1512</u> |
|                   | 36  | <b>1.7203</b> | <b>0.8030</b> | <b>1.1298</b> | 1.7415        | 0.8074        | 1.1368        | 1.7381        | 0.8067        | 1.1357        | <u>1.7332</u> | <u>0.8053</u> | <u>1.1341</u> | 1.7350        | 0.8059        | 1.1347        |
|                   | 48  | 1.6596        | 0.7944        | 1.0862        | <b>1.6238</b> | <b>0.7908</b> | <b>1.0745</b> | 1.6395        | 0.7940        | 1.0797        | 1.6602        | 0.7945        | 1.0864        | <u>1.6315</u> | <u>0.7927</u> | <u>1.0770</u> |
| Energy            | 12  | 1.0359        | 0.8831        | 2.0511        | <u>1.0010</u> | <u>0.8485</u> | <u>2.0163</u> | 1.1138        | 0.9270        | 2.1269        | 1.0330        | 0.8704        | 2.0484        | <b>0.9086</b> | <b>0.8070</b> | <b>1.9209</b> |
|                   | 24  | <u>1.3577</u> | <u>1.0415</u> | <u>2.3422</u> | 1.4262        | 1.0712        | 2.4005        | 1.3735        | 1.0480        | 2.3558        | <b>1.3544</b> | <b>1.0357</b> | <b>2.3394</b> | 1.4216        | 1.0731        | 2.3967        |
|                   | 36  | <b>1.5439</b> | <b>1.1228</b> | <b>2.4792</b> | 1.5599        | <u>1.1294</u> | 2.4920        | <u>1.5452</u> | 1.2281        | <u>2.4802</u> | 1.6140        | 1.1535        | 2.5349        | 1.5650        | 1.1316        | 2.4960        |
|                   | 48  | 1.6987        | 1.1829        | 2.5623        | 1.7437        | 1.2057        | 2.5960        | <u>1.6708</u> | <u>1.1734</u> | <u>2.5411</u> | 1.7444        | 1.2052        | 2.5966        | <b>1.6689</b> | <b>1.1729</b> | <b>2.5397</b> |
| Weather           | 48  | 0.4178        | 0.5438        | 0.7652        | 0.4241        | 0.5528        | 0.7710        | 0.4444        | 0.5588        | 0.7892        | 0.4500        | 0.5686        | 0.7942        | <b>0.4104</b> | <b>0.5398</b> | <b>0.7584</b> |
|                   | 96  | 0.6891        | 0.7212        | 1.0133        | <u>0.5645</u> | <u>0.6489</u> | <u>0.9172</u> | 0.7360        | 0.7487        | 1.0472        | 0.5977        | 0.6701        | 0.9438        | <b>0.5531</b> | <b>0.6281</b> | <b>0.9078</b> |
|                   | 192 | 0.8033        | 0.6863        | 1.1543        | <u>0.7761</u> | <u>0.6770</u> | <u>1.1345</u> | 0.8382        | 0.7029        | 1.1790        | 0.8422        | 0.7019        | 1.1819        | <b>0.7417</b> | <b>0.6681</b> | <b>1.1092</b> |
|                   | 336 | 0.7788        | 0.7560        | 1.0271        | <b>0.7526</b> | <b>0.7487</b> | <b>1.0096</b> | 0.7735        | 0.7532        | 1.0236        | <u>0.7620</u> | <u>0.7497</u> | <u>1.0160</u> | 0.7641        | 0.7541        | 1.0174        |
| Envi              | 48  | 0.4416        | 0.5275        | 1.0162        | 0.4419        | 0.5192        | 1.0136        | 0.4356        | 0.5090        | <u>1.0063</u> | 0.4603        | 0.5430        | 1.0345        | <b>0.4319</b> | <b>0.5004</b> | <b>1.0021</b> |
|                   | 96  | 0.4650        | 0.5416        | 1.0295        | 0.4737        | 0.5498        | 1.0391        | <b>0.4561</b> | <b>0.5190</b> | <b>1.0196</b> | 0.4730        | 0.5573        | 1.0384        | <u>0.4566</u> | <u>0.5331</u> | <u>1.0202</u> |
|                   | 192 | 0.4682        | 0.5214        | 1.0196        | 0.4630        | <u>0.5156</u> | <u>1.0139</u> | <b>0.4625</b> | <b>0.4995</b> | <b>1.0133</b> | 0.4724        | 0.5301        | 1.0241        | 0.4637        | 0.5180        | 1.0146        |
|                   | 336 | 0.5439        | 0.6071        | 1.0623        | 0.5668        | 0.6313        | 1.0844        | <u>0.5354</u> | <u>0.5975</u> | <u>1.0539</u> | 0.5586        | 0.6245        | 1.0765        | <b>0.5205</b> | <b>0.5829</b> | <b>1.0392</b> |
| Top-1 / Top-2     |     | 2 / 4         | 2 / 4         | 2 / 4         | 3 / 6         | 3 / 8         | 3 / 6         | 2 / 7         | 2 / 6         | 2 / 7         | 1 / 4         | 1 / 3         | 1 / 4         | <b>8 / 11</b> | <b>8 / 11</b> | <b>8 / 11</b> |
| Average Rank      |     | 3.0000        | 3.1250        | 3.0625        | 3.1250        | 2.8125        | 3.0625        | 3.1875        | 3.3750        | 3.1875        | 3.6250        | 3.5625        | 3.6250        | <b>2.0625</b> | <b>2.1250</b> | <b>2.0625</b> |



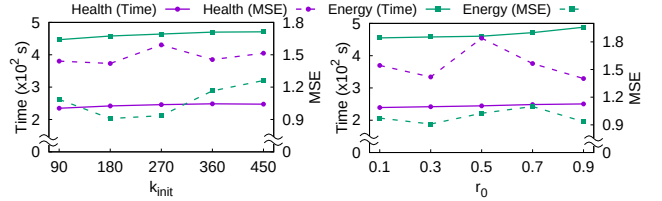
**Figure 10: Training time of Envi on varying different input lengths to the cross-modal attention.**



**Figure 11: Prediction MSE on Health varying  $N_1$  and  $N_2$ . Legends remain the same as Figure 10.**

historical information for forecasting. As input length increases, the MSE of all methods approaches the best performance under larger input lengths. Importantly, our method can process longer input lengths with the same runtime and obtain a lower MSE compared to other baselines. In addition, the reduction in time complexity introduced by our method does not result in degradation as input length changes, validating that our proposed attention mechanism focuses on important tokens rather than random pruning.

**Effect of selecting parameter  $k_{\text{init}}$  and  $r_0$**  Figure 12 shows the effect of selecting the initialized column subset size  $k_{\text{init}}$  and original Query matrix prune ratio  $r_0$  on Health and Energy with prediction length 12. These two parameters are highly relevant to selecting "informative" tokens. Training time shows only minor variation across different settings, indicating a stable computational cost under our adaptive column subset size selection Algorithm 7. Meanwhile, MSE fluctuates slightly within the tested ranges, suggesting the influence of varying these parameters is limited to LEO. In our experiments, we select the parameters that achieve the lowest MSE. Overall, the



**Figure 12: Parameter sensitivity varying  $k_{\text{init}}$  and  $r_0$ .**

results indicate that the proposed method remains stable without requiring careful hyperparameter tuning.

**KL divergence analysis.** Figure 13 reports the KL divergence as the max size of column subset  $k_{\text{max}}$  varies during training on Health and Energy with prediction length 48. As  $k_{\text{max}}$  increases, Skein shows a clear increase in KL divergence, indicating a growing distribution mismatch in attention weights. In contrast, our method maintains consistently low and stable KL values in all settings, demonstrating better preservation of the original attention distribution and lower sensitivity to  $k_{\text{max}}$ .

## 7.4 Ablation Study

To evaluate the effectiveness of attention distribution approximation with approximate histograms and Query pruning, we experiment with the prediction MSE and training time on Health here. Results on other datasets are similar and deferred to Appendix A.14. **Evaluation with and without pruning of the Query matrix.** Figure 14 evaluates the effect of query pruning and compares it with sparseVLM [69], a state-of-the-art token pruning method. For

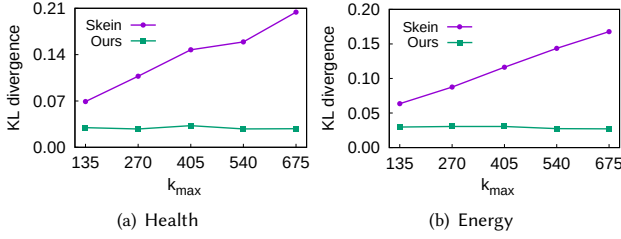


Figure 13: KL divergence varying  $k$ .

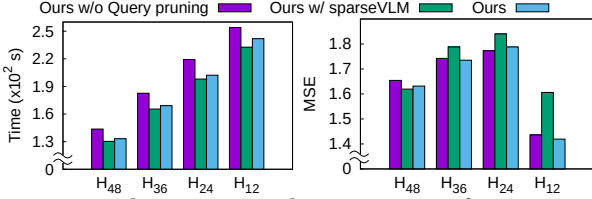


Figure 14: Prediction MSE and training time of Query matrix pruning on Health.

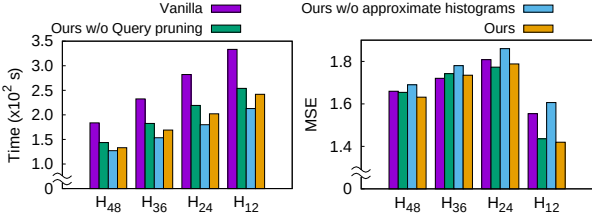


Figure 15: Prediction MSE and training time with/without approximate histograms on Health.

a fair comparison, we disable the token recycling component of sparseVLM, which is designed for inference-specific attention with deeper attention layers. First, our attention without Query matrix pruning consistently increases training time while having a similar MSE compared to our method, confirming the efficiency benefit. Second, we observe that using sparseVLM to replace our Query matrix pruning part could save some training time, but it leads to an unacceptably large MSE (e.g., from 1.42 to 1.61 for prediction length 12) because its Query pruning mechanism discards informative Query tokens. Thus, using our pruning strategy achieves the best balance between training efficiency and prediction accuracy.

**Evaluation with and without approximate histograms.** Figure 15 presents the impact of CSS with approximate histograms on Health. Our attention without approximate histograms obtains worse prediction MSE while reducing training time, indicating an effective attention approximation. In contrast, our attention achieves a relatively short training time and a stable prediction MSE, benefiting from the optimization in the approximation. Overall, our method achieves a better efficiency–accuracy trade-off.

## 8 RELATED WORK

In this section, we review two lines of work: multimodal time series forecasting with cross-modal attention and efficient Transformers. **Multimodal time series forecasting with cross-modal attention.** Time series forecasting predicts future values from historical data [7, 33, 37, 38, 55, 61], while multimodal forecasting extends this

paradigm by incorporating auxiliary modalities such as text and images. Recent studies [44, 58, 72] show that language signals provide high-level context for non-stationary trends and rare events, while visual cues inject semantic priors difficult to infer from time series alone. To capture cross-modal dependencies, existing methods adopt early, intermediate, or late fusion strategies [34, 71, 72].

Among these approaches, cross-modal attention is widely used for early or intermediate fusion, enabling fine-grained interactions between modalities. Previous work applies cross-attention for representation integration [28, 48] or introduces modality-aware designs to address relevance and temporal misalignment [67]. However, dense cross-modal attention incurs high computational overhead and scales poorly with sequence length and the number of modalities. Existing optimizations such as EMT [45] remain quadratic. This highlights the need for more efficient cross-modal attention mechanisms for multimodal time series forecasting.

**Efficient transformers.** A large body of work accelerates transformers by reducing the quadratic cost of attention, primarily for unimodal settings. Kernel-based methods approximate softmax attention via kernel feature maps and obtain linear-time attention [8, 75]. Locality-aware methods restrict attention to a subset of tokens via content-based grouping or hashing; Reformer, for example, applies LSH to limit each query to a small candidate set [25]. Randomized and sketching-based techniques approximate attention matrices through sampling or low-dimensional projections [5, 73].

Despite their success, these techniques are often not directly applicable to cross-modal attention because  $Q$  and  $K/V$  originate from different modalities with distinct distributions and dimensions. This breaks key assumptions (e.g., shared feature space) and complicates approximation design, motivating efficiency mechanisms tailored to cross-modal attention. Recent works on efficient attention primarily focus on accelerating LLM inference [6, 46], relying on inference-specific mechanisms such as KV caching [26, 63, 66]. In contrast, our work studies efficient cross-modal attention during training. Since cross-modal attention builds upon the standard attention formulation, we adapt representative efficient attention mechanisms to the cross-modal setting as baselines for comparison.

## 9 CONCLUSION

In this paper, we propose the first efficient cross-modal attention mechanism, LEO, a data-driven optimization for training cross-modal attention mechanisms that reduces the time complexity from quadratic to linearithmic. The experiment shows that our method is more efficient than all compared methods and achieves nearly the best average rank in terms of prediction results. While LEO focuses on efficiency, it achieves competitive memory usage compared to other baselines. Moreover, LEO can also be incorporated with other transformer architectures, e.g., Flash attentions [9, 10, 65] to further optimize the space complexity with minor modifications. In the future, we plan to apply LEO to optimize other multimodal downstream tasks and extend it to VLMs with masks.

## AI/LLM USAGE DISCLOSURE

ChatGPT was used to polish the language for better readability. All technical contributions, including the proposed method, analysis, and experiments, were developed and verified by the authors.

## REFERENCES

- [1] Jayadev Acharya, Ilias Diakonikolas, Chinmay Hegde, Jerry Zheng Li, and Ludwig Schmidt. 2015. Fast and near-optimal algorithms for approximating distributions by histograms. In *Proceedings of the 34th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 249–263.
- [2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35 (2022), 23716–23736.
- [3] Jason Altschuler, Aditya Bhaskara, Gang Fu, Vahab Mirrokni, Afshin Rostamizadeh, and Morteza Zadimoghaddam. 2016. Greedy column subset selection: New bounds and distributed algorithms. In *International conference on machine learning*. PMLR, 2539–2548.
- [4] Christos Boutsidis, Michael W Mahoney, and Petros Drineas. 2009. An improved approximation algorithm for the column subset selection problem. In *Proceedings of the twentieth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 968–977.
- [5] Yifan Chen, Qi Zeng, Dilek Hakkani-Tur, Di Jin, Heng Ji, and Yun Yang. 2022. Sketching as a tool for understanding and accelerating self-attention for long sequences. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 5187–5199.
- [6] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, and Beidi Chen. 2025. MagicPIG: LSH Sampling for Efficient LLM Generation. In *The Thirteenth International Conference on Learning Representations*.
- [7] Yunyao Cheng, Chenjuan Guo, Bin Yang, Haomin Yu, Kai Zhao, and Christian S Jensen. 2024. A Memory Guided Transformer for Time Series Forecasting. *Proceedings of the VLDB Endowment* 18, 2 (2024), 239–252.
- [8] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. 2020. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794* (2020).
- [9] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [10] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [11] Amit Deshpande and Luis Rademacher. 2010. Efficient Volume Sampling for Row/Column Subset Selection. In *2010 IEEE 51st Annual Symposium on Foundations of Computer Science*. IEEE, 329–338.
- [12] Alin Dobra. 2005. Histograms revisited: when are histograms the best approximation method for aggregates over joins?. In *Proceedings of the twenty-fourth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 228–237.
- [13] Petros Drineas, Michael W Mahoney, and Shan Muthukrishnan. 2008. Relative-error CUR matrix decompositions. *SIAM J. Matrix Anal. Appl.* 30, 2 (2008), 844–881.
- [14] Dafydd Evans, Antonia J Jones, and Wolfgang M Schmidt. 2002. Asymptotic moments of near-neighbour distance distributions. *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences* 458, 2028 (2002), 2839–2849.
- [15] Ahmed K Farahat, Ahmed Elgohary, Ali Ghodsi, and Mohamed S Kamel. 2015. Greedy column subset selection for large-scale data sets. *Knowledge and Information Systems* 45 (2015), 1–34.
- [16] Sudipto Guha, Nick Koudas, and Kyuseok Shim. 2001. Data-streams and histograms. In *Proceedings of the thirty-third annual ACM symposium on Theory of computing*. 471–475.
- [17] Sudipto Guha, Nick Koudas, and Kyuseok Shim. 2006. Approximation and streaming algorithms for histogram construction problems. *ACM Transactions on Database Systems (TODS)* 31, 1 (2006), 396–438.
- [18] Venkatesan Guruswami and Ali Kemal Sinop. 2012. Optimal column-based low-rank matrix reconstruction. In *Proceedings of the twenty-third annual ACM-SIAM symposium on Discrete Algorithms*. SIAM, 1207–1214.
- [19] Zengfeng Huang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2017. Efficient matrix sketching over distributed data. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 347–359.
- [20] Piotr Indyk, Reut Levi, and Ronitt Rubinfeld. 2012. Approximating and testing k-histogram distributions in sub-linear time. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI symposium on Principles of Database Systems*. 15–22.
- [21] Yannis Ioannidis. 2003. The history of histograms (abridged). In *Proceedings 2003 VLDB Conference*. Elsevier, 19–30.
- [22] Kaixiang Ji, Jiajia Liu, Weixiang Hong, Liheng Zhong, Jian Wang, Jingdong Chen, and Wei Chu. 2022. Cret: Cross-modal retrieval transformer for efficient text-video retrieval. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 949–959.
- [23] Kai Kim, Howard Tsai, Rajat Sen, Abhimanyu Das, Zihao Zhou, Abhishek Tanpure, Mathew Luo, and Rose Yu. 2024. Multi-modal forecaster: Jointly predicting time series and textual data. *arXiv preprint arXiv:2411.06735* (2024).
- [24] Sehoon Kim, Amir Gholami, Albert Shaw, Nicholas Lee, Karttikeya Mangalam, Jitendra Malik, Michael W Mahoney, and Kurt Keutzer. 2022. Squeezeformer: An efficient transformer for automatic speech recognition. *Advances in Neural Information Processing Systems* 35 (2022), 9361–9373.
- [25] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. 2020. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451* (2020).
- [26] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. 2025. Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems. *Proceedings of the ACM on Management of Data* 3, 4 (2025), 1–27.
- [27] Jiaming Liang, Lei Cao, Samuel Madden, Zack Ives, and Guoliang Li. 2024. RITA: Group Attention is All You Need for Timeseries Analytics. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–28.
- [28] Junyang Lin, An Yang, Yichang Zhang, Jie Liu, Jingren Zhou, and Hongxia Yang. 2020. Interbert: Vision-and-language interaction for multi-modal pretraining. *arXiv preprint arXiv:2003.13198* (2020).
- [29] Haoxin Liu, Shangqing Xu, Zhiyuan Zhao, Linghai Kong, Harshvardhan Prabhakar Kamarthi, Aditya Sasanur, Megha Sharma, Jiaming Cui, Qingsong Wen, Chao Zhang, et al. 2024. Time-mmd: Multi-domain multimodal dataset for time series analysis. *Advances in Neural Information Processing Systems* 37 (2024), 77888–77933.
- [30] Andrzej Maćkiewicz and Waldemar Ratajczak. 1993. Principal components analysis (PCA). *Computers & Geosciences* 19, 3 (1993), 303–342.
- [31] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [32] Arsha Nagrani, Shan Yang, Anurag Arnab, Aren Jansen, Cordelia Schmid, and Chen Sun. 2021. Attention bottlenecks for multimodal fusion. *Advances in neural information processing systems* 34 (2021), 14200–14213.
- [33] Yiming Niu, Jinliang Deng, Lulu Zhang, Zimu Zhou, and Yongxin Tong. 2025. Accurate and Efficient Multivariate Time Series Forecasting via Offline Clustering. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3056–3069.
- [34] Dimitrios Oikonomou, Lampros Leontaris, Nikolaos Dimitriou, and Dimitrios Tzovaras. 2025. Time-series forecasting in industrial environments: A performance study and a novel late fusion framework. *IEEE Sensors Journal* 25, 4 (2025), 7681–7697.
- [35] Bruno Ordozgoiti, Sandra Gómez Canaval, and Alberto Mozo. 2018. Iterative column subset selection. *Knowledge and Information Systems* 54 (2018), 65–94.
- [36] Dimitris Papailiopoulos, Anastasios Kyriklidis, and Christos Boutsidis. 2014. Provable deterministic leverage score sampling. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 997–1006.
- [37] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S Jensen, Zhenli Sheng, et al. 2024. TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods. *Proceedings of the VLDB Endowment* 17, 9 (2024), 2363–2377.
- [38] Xiangfei Qiu, Xiuwen Li, Ruiyang Pang, Zhicheng Pan, Xingjian Wu, Liu Yang, Jilin Hu, Yang Shu, Xuesong Lu, Chengcheng Yang, et al. 2025. Easytime: Time series forecasting made easy. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 4564–4567.
- [39] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- [40] Sidney I Resnick. 2007. *Heavy-tail phenomena: probabilistic and statistical modeling*. Vol. 10. Springer Science & Business Media.
- [41] Yasunari Shidama. 2004. The Taylor expansions. *Formalized Mathematics* 12, 2 (2004), 195–200.
- [42] Yaroslav Shitov. 2021. Column subset selection is NP-complete. *Linear Algebra Appl.* 610 (2021), 52–58.
- [43] Bernard W Silverman. 2018. *Density estimation for statistics and data analysis*. Routledge.
- [44] Chen Su, Yuanhe Tian, Yan Song, and Yongdong Zhang. 2025. Text reinforcement for multimodal time series forecasting. *arXiv preprint arXiv:2509.00687* (2025).
- [45] Licai Sun, Zheng Lian, Bin Liu, and Jianhua Tao. 2023. Efficient multimodal transformer with dual-level feature restoration for robust multimodal sentiment analysis. *IEEE Transactions on Affective Computing* 15, 1 (2023), 309–325.
- [46] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*. 47901–47911.
- [47] Nitin Thaper, Sudipto Guha, Piotr Indyk, and Nick Koudas. 2002. Dynamic multidimensional histograms. In *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*. 428–439.

- [48] Yao-Hung Hubert Tsai, Shaojie Bai, Paul Pu Liang, J Zico Kolter, Louis-Philippe Morency, and Ruslan Salakhutdinov. 2019. Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of the conference. Association for computational linguistics. Meeting*, Vol. 2019. 6558.
- [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [50] Meng Wang, Jintao Yang, Bin Yang, Hui Li, Tongxin Gong, Bo Yang, and Jiangtao Cui. 2025. Towards Lightweight Time Series Forecasting: A Patch-Wise Transformer with Weak Data Enriching. In *2025 IEEE 41st International Conference on Data Engineering*. 1278–1291. <https://doi.org/10.1109/ICDE65448.2025.00100>
- [51] Ningning Wang, Guobing Gan, Peng Zhang, Shuai Zhang, Junqiu Wei, Qun Liu, and Xin Jiang. 2022. ClusterFormer: Neural clustering attention for efficient and effective transformer. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2390–2402.
- [52] Tiannan Wang, Wangchunshu Zhou, Yan Zeng, and Xinsong Zhang. 2023. Efficientvlm: Fast and accurate vision-language models via knowledge distillation and modal-adaptive pruning. In *Findings of the Association for Computational Linguistics: ACL 2023*. 13899–13913.
- [53] Zichen Wen, Yifeng Gao, Weijia Li, Conghui He, and Linfeng Zhang. 2025. Token pruning in multimodal large language models: Are we solving the right problem?. In *Findings of the Association for Computational Linguistics: ACL 2025*. 15537–15549.
- [54] Xingjian Wu, Jianxin Jin, Wanghui Qiu, Peng Chen, Yang Shu, Bin Yang, and Chenjuan Guo. 2026. Aurora: Towards Universal Generative Multimodal Time Series Forecasting. In *ICLR*.
- [55] Xinle Wu, Xingjian Wu, Dalin Zhang, Miao Zhang, Chenjuan Guo, Bin Yang, and Christian S. Jensen. 2024. Fully Automated Correlated Time Series Forecasting in Minutes. *Proc. VLDB Endow.* 18, 2 (Oct. 2024), 144–157. <https://doi.org/10.14778/3705829.3705835>
- [56] Haojun Xia, Zhen Zheng, Yuchao Li, Donglin Zhuang, Zhongzhu Zhou, Xiafei Qiu, Yong Li, Wei Lin, and Shuaiwen Leon Song. 2023. Flash-LLM: Enabling Cost-Effective and Highly-Efficient Large Generative Model Inference with Unstructured Sparsity. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 211–224. <https://doi.org/10.14778/3626292.3626303>
- [57] Lianghao Xia, Chao Huang, and Chuxu Zhang. 2022. Self-supervised hypergraph transformer for recommender systems. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 2100–2109.
- [58] Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. 2025. ChatTS: Aligning Time Series with LLMs via Synthetic Data for Enhanced Understanding and Reasoning. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2385–2398.
- [59] Derong Xu, Jingbo Zhou, Tong Xu, Yuan Xia, Ji Liu, Enhong Chen, and De-jing Dou. 2023. Multimodal biological knowledge graph completion via triple co-attention mechanism. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 3928–3941.
- [60] Dezhong Yao, Yuhong Gu, Gao Cong, Hai Jin, and Xinqiao Lv. 2022. Entity Resolution with Hierarchical Graph Attention Networks. In *ACM SIGMOD, Zachary G. Ives, Angela Bonifati, and Amr El Abbadi (Eds.)*. 429–442.
- [61] Hengyu Ye, Jiadong Chen, Fuxin Jiang, Xiao He, Tieying Zhang, Jianjun Chen, and Xiaofeng Gao. 2025. Framer: Lightweight and Effective Frequency Transformer for Workload Forecasting in Cloud Services. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3812–3825.
- [62] Hanyan Yin, Dongxue Wen, Jiajun Li, Zhewei Wei, Xiao Zhang, Peng Zhao, and Zhi-Hua Zhou. 2026. AeroSketch: Near-Optimal Time Matrix Sketch Framework for Persistent, Sliding Window, and Distributed Streams. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 49.
- [63] Hao Yuan, Xin Ai, Qiange Wang, Peizheng Li, Jiayang Yu, Chaoyi Chen, Xinbo Yang, Yanfeng Zhang, Zhenbo Fu, Yingyou Wen, et al. 2025. DepCache: A KV Cache Management Framework for GraphRAG with Dependency Attention. *Proceedings of the ACM on Management of Data* 3, 6 (2025), 1–29.
- [64] Haitao Yuan, Gao Cong, and Guoliang Li. 2024. Nuhuo: An effective estimation model for traffic speed histogram imputation on A road network. *Proceedings of the VLDB Endowment* 17, 7 (2024), 1605–1617.
- [65] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. 2026. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. *arXiv preprint arXiv:2603.05451* (2026).
- [66] Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. 2025. Pqcache: Product quantization-based kvcache for long context llm inference. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–30.
- [67] Lige Zhang, Ali Maatouk, Jialin Chen, Leandros Tassioulas, and Rex Ying. 2026. Multi-Modal Time Series Prediction via Mixture of Modulated Experts. *arXiv preprint arXiv:2601.21547* (2026).
- [68] Xindong Zhang, Hui Zeng, Shi Guo, and Lei Zhang. 2022. Efficient long-range attention network for image super-resolution. In *European conference on computer vision*. Springer, 649–667.
- [69] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. 2025. SparseVLM: Visual Token Sparsification for Efficient Vision-Language Model Inference. In *International Conference on Machine Learning*. 74840–74857.
- [70] Ziqian Zhang, Chaokun Wang, Shuwen Zheng, Cheng Wu, Ziyang Liu, and Hao Feng. 2025. Effective and Scalable Heterogeneous Graph Neural Network Framework with Convolution-oriented Attention. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 4011–4024. <https://doi.org/10.1109/ICDE65448.2025.00299>
- [71] Taibiao Zhao, Xiaobing Chen, and Mingxuan Sun. 2025. Enhancing time series forecasting via multi-level text alignment with llms. In *International Conference on Database Systems for Advanced Applications*. 507–516.
- [72] Siru Zhong, Weilin Ruan, Ming Jin, Huan Li, Qingsong Wen, and Yuxuan Liang. 2025. Time-vlm: Exploring multimodal vision-language models for augmented time series forecasting. *arXiv preprint arXiv:2502.04395* (2025).
- [73] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.
- [74] Lei Zhu, Chaoqun Zheng, Weili Guan, Jingjing Li, Yang Yang, and Heng Tao Shen. 2023. Multi-modal hashing for efficient multimedia retrieval: A survey. *IEEE Transactions on Knowledge and Data Engineering* 36, 1 (2023), 239–260.
- [75] Rundong Zuo, Guozhong Li, Rui Cao, Byron Choi, Jianliang Xu, and Sourav S Bhowmick. 2024. DARKER: Efficient Transformer with Data-Driven Attention Mechanism for Time Series. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3229–3242.
- [76] Rundong Zuo, Guozhong Li, Byron Choi, Sourav S Bhowmick, Daphne Ngar-yin Mah, and Grace LH Wong. 2023. SVP-T: A shape-level variable-position transformer for multivariate time series classification. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11497–11505.

## A APPENDIX

### A.1 Additional Analyses on Cross-modal Attention

Figure 16 shows the "interactions" between modalities  $A$  and  $B$  and the structure of a vanilla cross-modal Transformer layer. In multimodal Transformers, cross-modal attention is typically applied in both directions:

$$\begin{aligned} X_{b \rightarrow a}^{(L)} &= \text{CMT\_Layer}_{b \rightarrow a}(X_a, X_b), \\ X_{a \rightarrow b}^{(L)} &= \text{CMT\_Layer}_{a \rightarrow b}(X_b, X_a), \end{aligned} \quad (22)$$

where  $\text{CMT\_Layer}_{b \rightarrow a}(X_a, X_b)$  represents that  $X_a$  is transformed to be Query matrix shown as *Cross-modal Transformer Layer* in Figure 16. Noted that A single-direction cross-modal attention is employed when the target modality (e.g., modality A in Equation 1) is determined by downstream tasks.

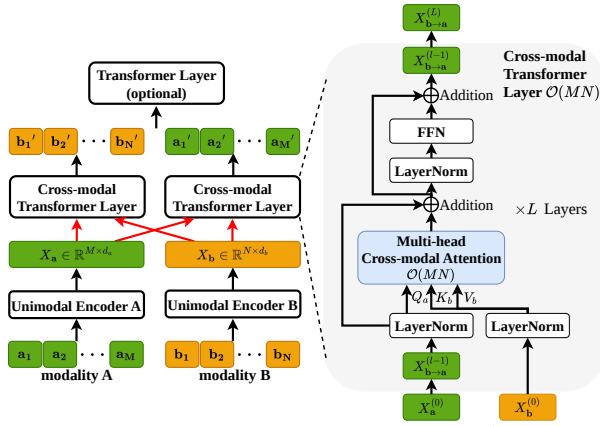


Figure 16: Cross-modal attention interaction (indicated in red) between modalities  $A$  and  $B$  and structure of the vanilla attention layer [48].

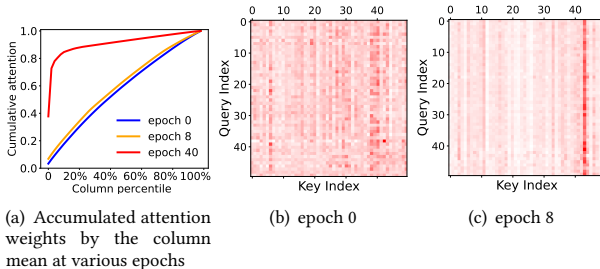


Figure 17: Long-tailed distribution of an attention matrix (upper) and cross-modal attention patterns of Column-wise Attention Concentration in different epochs (bottom) with MulT on CMU-MOSI dataset

### A.2 Additional Details of the Cost Model

We explain the trade-off of  $k$  between efficiency and accuracy of AttnApprox. On one hand, a larger size of the selected column subset indicates less error for the Key matrix approximation and a smaller difference between the distribution of our method and the vanilla attention, as illustrated in Equation 15 (Theorem 6.1). However, it leads to a higher computational cost of LEO and a higher index dimension required for the index  $\mathcal{I}$ , which also decreases the

efficiency as analyzed in the time complexity of AttnApprox. On the other hand,  $k$  is relative to our search vector filtering since it can control the true skewed vector ratio  $v$ . In particular, some head vectors  $\tilde{a}_{j \in R}$  have already approached the tail of the distribution and thus need no detailed approximation with histograms. Therefore,  $k$  requires adjustment when the number of filtered queries is too small to incur extra cost during the later search.

Figure 25(a) shows the relationships among column subset size  $k$ , skewed vector ratio  $v$  and search time of each vector, validating that the time cost of each search grows linearly with  $k$ , and meanwhile the skewed ratio  $v$  decreases exponentially under a fixed threshold  $\theta$ . Figure 25(b) illustrates that total search time decreases simultaneously with the skewed ratio  $v$  when increasing  $\theta$ . Combining with Equation 17, the two figures demonstrate that a trade-off between efficiency and KL divergence exists when reducing  $k$  in the training. Additionally, it is difficult to quantify the functional relationship between the variable  $N_\theta$  and  $k$ , which indicates that minor optimizing is a practical and robust scheme rather than directly selecting the optimal via functions like  $\arg\min(\cdot)$ .

Here we present Figure 25(c) of the relationship between the skewed vector ratio  $v$  and column subset size  $k$  to validate our adaptation algorithm. In practice, we regulate  $v$  to remain within a predefined skewed range  $[v_{\min}, v_{\max}]$ , which avoids the search being ineffective (smaller  $v$ ) or time-consuming (larger  $v$ ). It shows that optimizing  $k$  and  $\theta$  together can effectively make the optimal  $k^*$  lie in the skewed range, validating the effectiveness of Algorithm 7.

### A.3 Supplementary Algorithms

---

#### Algorithm 4: REEstimate( $Q, K, Y$ )

---

**Input:** Query matrix  $Q$ , Key matrix  $K$ , attention output  $Y$  from last iteration

**Output:** Query prune ratio  $r_p$  for the current iteration

- 1 Compute representations  $a, b, y$  with input  $Q, K, Y$ ;
- 2  $c_a \leftarrow |a^\top y|$ ; // Compute relevance
- 3  $c_b \leftarrow |b^\top y|$ ;
- 4  $S_A \leftarrow \frac{c_a}{c_a + c_b}$ ; // Compute batch-wise prune ratio
- 5  $r_p \leftarrow 1 - S_A$ ;
- 6 **return**  $r_p$

---



---

#### Algorithm 5: IndexBuild( $A_{\text{sample}}, A_{\text{head}}, I_\mu, \{I_\nu^t\}_{t=1}^{T+1}$ )

---

**Input:** Sampled attention  $A_{\text{sample}}$ , attention for searching  $A_{\text{head}}$ , sampled Query vector IDs  $I_\mu$ , selected Key vector ID subsets  $\{I_\nu^t\}_{t=1}^{T+1}$

**Output:** Vector index  $\mathcal{I}$ , hashmap of histograms  $\mathcal{T}$

- 1 Initialize an empty vector index  $\mathcal{I}$  and hashmap  $\mathcal{T}$ ;
- 2 **for**  $i \in I_\mu$  **do**
- 3     compute  $(s_{\text{nor}}^i, \dots, b_T^i, \dots, b_T^i)$  from  $A_{\text{sample}}$  with  $\{I_\nu^t\}_{t=1}^{T+1}$ ; // Eq. 14
- 4     store  $(s_{\text{nor}}^i, \dots, b_T^i)$  in hashmap  $\mathcal{T}$  for histogram bucket mean search;
- 5 compute default value  $(s_{\text{nor}}^0, \dots, b_T^0)$  from  $s_{\text{nor}}$ ;
- 6 store default value  $(s_{\text{nor}}^0, \dots, b_T^0)$  in  $\mathcal{T}$  at index 0;
- 7  $\mathcal{I}.\text{insert}(\text{sort}(A_{\text{head}}[I_\mu, :], \text{desc}));$
- 8 **return**  $(\mathcal{I}, \mathcal{T})$

---

---

**Algorithm 6:** AttnApprox( $A_{\text{head}}, \mathcal{I}, \mathcal{T}, h, \theta$ )

---

**Input:** Attention  $A_{\text{head}}$  for search, vector index  $\mathcal{I}$ , hashmap  $\mathcal{T}$ , vector split ratio  $h$ , variance contribution ratio threshold  $\theta$   
**Output:** Approximate attention after normalization  $A_{\text{apx}}$  and histogram bucket means  $(b_1, \dots, b_T)$  to approximate the tail distribution, true skewed vector ratio  $v$

```
1 // (1) Search vector filtering with variance contribution ratio
2  $R \leftarrow \emptyset$ ; // Initialize  $R$  for skewed vectors
3 for  $a_j = A_{\text{head}}[j, :]$  do
4    $\tilde{a}_j \leftarrow \text{sort}(a_j, \text{desc})$ ;
5    $\tilde{a}_j^{(I)} \leftarrow \tilde{a}_j[h|I_v|:]$ ,  $\tilde{a}_j^{(II)} \leftarrow \tilde{a}_j[h|I_v|:]$ ;
6    $\vartheta_j \leftarrow \|\tilde{a}_j^{(I)}\|_2^2 / \|\tilde{a}_j^{(II)}\|_2^2$ ;
7   if  $\vartheta_j > \theta$  then
8      $R.add(j)$ ; // Keep skewed vectors
9  $v \leftarrow |R| / \text{len}(A_{\text{pre}})$ ; // Compute the skewed vector ratio
10 // (2) Histogram bucket mean search
11 Initialize  $w_j$  of  $\tilde{a}_j$  to 0; // the position of the default entry in  $\mathcal{T}$ 
12 for each row  $j$  in  $A_{\text{head}}$  do
13   if  $j \in R$  then
14      $w_j \leftarrow \mathcal{I}.\text{search}(\tilde{a}_j)$ ;
15      $(s_{\text{nor}}^j, \dots, s_T^j) \leftarrow \mathcal{T}.\text{query}(w_j)$ ;
16      $A_{\text{apx}}[j, :] \leftarrow s_{\text{nor}}^j A_{\text{head}}[j, :]$ ; // Normalize attention row
17 return ( $A_{\text{apx}}, (b_1, \dots, b_T), v$ );
```

---

---

**Algorithm 7:** ColAdapt( $k, v, \theta, \tau$ )

---

**Input:** Column subset size  $k$ , skewed vector ratio  $v$ , variance contribution ratio threshold  $\theta$  from last iteration, target ratio  $\tau$   
**Output:** Column subset size  $k$ , variance contribution ratio threshold  $\theta$  for the current iteration

```
1  $e \leftarrow v - \tau$ ; // Compute error  $e$  to target ratio  $\tau$ 
2  $\theta \leftarrow \theta \cdot \exp(e)$ ;
3  $k \leftarrow k - \lfloor e \cdot k \rfloor$ ;
4 return ( $k, \theta$ );
```

---

#### A.4 Proof of Theorem: Softmax Suppression of Small Values

**THEOREM A.1 (SOFTMAX SUPPRESSION OF SMALL VALUES).** Assume an attention logit matrix before softmax  $e = [e_1, \dots, e_M]^\top \in \mathbb{R}^{M \times N}$ . Let  $e_i \in \mathbb{R}^{1 \times N}$  be the  $i$ -th row of  $e$  and  $A_i = \text{softmax}(e_i)$ . Assume the logits  $e_{ij}$  are uniformly bounded: there exists a constant  $\zeta > 0$ , such that  $|e_{ij}| \leq \zeta$ ,  $\forall j \in \{1, \dots, N\}$ . Then,

$$\lim_{N \rightarrow \infty} \min_{1 \leq j \leq N} a_{ij} = 0. \quad (23)$$

**PROOF.** Fix  $N$ . By the boundedness assumption, for every  $j$  we have

$$\exp(e_{ij}) \leq \exp(\zeta), \quad (24)$$

and for every  $k$  we have

$$\exp(e_{ik}) \geq \exp(-\zeta). \quad (25)$$

Therefore, the softmax denominator satisfies

$$\sum_{k=1}^N \exp(e_{ik}) \geq \sum_{k=1}^N \exp(-\zeta) = N \exp(-\zeta). \quad (26)$$

Combining the numerator and denominator bounds yields, for all  $j \in \{1, \dots, N\}$ ,

$$a_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^N \exp(e_{ik})} \leq \frac{\exp(\zeta)}{N \exp(-\zeta)} = \frac{\exp(2\zeta)}{N}. \quad (27)$$

Taking the minimum over  $j$  preserves the upper bound:

$$\min_{1 \leq j \leq N} a_{ij} \leq \frac{\exp(2\zeta)}{N}. \quad (28)$$

Since  $\exp(2\zeta)$  is constant and  $\exp(2\zeta)/N \rightarrow 0$  as  $N \rightarrow \infty$ , the squeeze principle implies

$$\lim_{N \rightarrow \infty} \min_{1 \leq j \leq N} a_{ij} = 0, \quad (29)$$

which concludes the proof.  $\square$

#### A.5 Proof of Theorem: Bounded Ratio Between $E_1$ and $E_2$

**THEOREM A.2 (BOUNDED ERROR OF  $E_{\text{CSS-MUL}}$  BY  $E_{\text{MUL-CSS}}$ ).** Let  $Q \in \mathbb{R}^{M \times d}$  and  $K^\top \in \mathbb{R}^{d \times N}$ , and let  $K^\top = U\Sigma V^\top$  be the singular value decomposition of  $K^\top$ , where  $U = [u_1, \dots, u_r]$  and  $r = \text{rank}(K)$ . Assume that  $Q^\top Q$  is diagonal in the basis  $\{u_i\}_{i=1}^r$ , i.e.,  $u_i^\top Q^\top Q u_j = 0$ , for  $i \neq j$ , and define  $\alpha := \min_{i>k} \|Qu_i\|_2 > 0$ . Then

$$E_{\text{CSS-Mul}} \leq \frac{\|Q\|_2^2}{\alpha^2} (1 + \epsilon) E_{\text{Mul-CSS}}. \quad (30)$$

**PROOF.** Let  $C$  denote the column subset selected by Algorithm 1. Let  $P_C := CC^\top$  denote the projection onto the span of the selected columns. By the construction of Algorithm 1 and its connection to CSS, the selected subset  $C$  satisfies the standard relative-error guarantee [11],

$$\|K^\top - P_C K^\top\|_F^2 \leq (1 + \epsilon) \|K^\top - K_k^\top\|_F^2. \quad (31)$$

Since

$$QK^\top = Q(U\Sigma V^\top) = \sum_{i=1}^r \sigma_i (Qu_i) v_i^\top,$$

and  $u_i^\top Q^\top Q u_j = 0$  for  $i \neq j$ , the vectors  $\{Qu_i\}_{i=1}^r$  are mutually orthogonal. Hence

$$\|QK^\top - (QK^\top)_k\|_F^2 = \sum_{i>k} \sigma_i^2 \|Qu_i\|_2^2.$$

Therefore,

$$E_{\text{Mul-CSS}} = \sum_{i>k} \sigma_i^2 \|Qu_i\|_2^2 \geq \alpha^2 \sum_{i>k} \sigma_i^2 = \alpha^2 \|K^\top - K_k^\top\|_F^2.$$

On the other hand,

$$E_{\text{CSS-Mul}} = \|Q(K^\top - CC^\top K^\top)\|_F^2 \leq \|Q\|_2^2 \|K^\top - CC^\top K^\top\|_F^2.$$

Using the relative-error guarantee of CSS, we obtain

$$E_{\text{CSS-Mul}} \leq \|Q\|_2^2 (1 + \epsilon) \|K^\top - K_k^\top\|_F^2.$$

Combining the above two inequalities yields

$$E_{\text{CSS-Mul}} \leq \frac{\|Q\|_2^2}{\alpha^2} (1 + \epsilon) E_{\text{Mul-CSS}}. \quad \square$$

#### A.6 Proof of Theorem: Error of Pruning Query Vectors

**THEOREM A.3 (ERROR OF QUERY MATRIX PRUNING).** Given the  $i$ -th gate score  $g_i$  of learned gate  $g$ , Value matrix  $V$ , and its column mean  $\bar{V}$ . If we obtain the  $i$ -th row of the attention weight matrix  $A_i$  with vanilla attention, the error  $E_{\text{project}}$  of the attention output in the

$i$ -th row brought by Query matrix pruning satisfies:

$$E_{\text{project}} = \|g_i \cdot A_i \cdot V - g_i \cdot \tilde{V}\|_F \leq 2g_i \|V\|_F. \quad (32)$$

PROOF. Since  $A_i$  is attention weights normalized by Softmax( $\cdot$ ), its norm always satisfies  $\|A_i\|_F \leq 1$ . Thus, according to the triangle inequality, we have

$$\begin{aligned} E_{\text{project}} &= \|g_i \cdot A_i \cdot V - g_i \tilde{V}\|_F \leq \|g_i \cdot A_i \cdot V\|_F + \|g_i \tilde{V}\|_F \\ &\leq g_i \cdot (\|A_i V\|_F + \|\tilde{V}\|_F) \leq g_i \cdot (\|V\|_F + \|V\|_F) = 2g_i \|V\|_F \end{aligned} \quad (33)$$

□

## A.7 Proof of Theorem: KL Divergence Expectation

**THEOREM A.4 (KL DIVERGENCE EXPECTATION).** Consider one row distribution of an attention matrix after softmax, where each  $p_i = \frac{e^{b_i}}{Z}$ ,  $Z = \sum_{j=1}^N e^{b_j}$ , sorted in decreasing order of attention weight. Denote  $\hat{p}_i$  by our distribution in the same row. Assume attention weights in one row follow a truncated power law  $p_{(i)} = \frac{i^{-\alpha}}{H_{N,\alpha}}$ ,  $\alpha > 1$  where  $H_{N,\alpha} = \sum_{j=1}^N j^{-\alpha}$ . The KL divergence between  $p_i$  and  $\hat{p}_i$  satisfies the following

$$\mathbb{E}[\text{KL}(p \|\hat{p})] = \Theta(\log N \cdot k^{1-\alpha}). \quad (34)$$

PROOF. For convenience, we present the proof under  $T = 2$ , and larger  $T$  have similar results. We consider one row of an attention matrix after softmax, where each  $p_i = \frac{e^{b_i}}{Z}$ ,  $Z = \sum_{j=1}^N e^{b_j}$ , sorted in decreasing order of attention weight. As illustrated in AttnApprox, we partition each row into three groups:  $T = \{1, \dots, k\}$ ,  $U = \{k+1, \dots, 2k\}$ ,  $R = \{2k+1, \dots, N\}$ , representing distributional head, bucket 1, and bucket 2 to compute  $s_{\text{nor}}, b_1, b_2$ . Let group masses be  $P_T = \sum_{i \in T} p_i$ ,  $P_U = \sum_{i \in U} p_i$ ,  $P_R = \sum_{i \in R} p_i$ . The approximation  $\hat{p}$  is constructed as: (a) for  $i \in T$ ,  $\hat{p}_i = s_{\text{nor}} q_i$ ,  $q_i = \frac{e^{b_i}}{\sum_{j \in T} e^{b_j}}$  preserving relative structure inside top- $k$ ; (b) for  $i \in U$ ,  $\hat{p}_i = b_1$ ; (c) for  $i \in R$ ,  $\hat{p}_i = b_2$ . Define approximate group masses  $\hat{P}_T = s_{\text{nor}}$ ,  $\hat{P}_U = kb_1$ ,  $\hat{P}_R = (N-2k)b_2$ . Since  $\hat{p}$  is normalized so we have  $\hat{P}_T + \hat{P}_U + \hat{P}_R = 1$ . The KL divergence has the decomposition

$$\begin{aligned} \text{KL}(p \|\hat{p}) &= \sum_{g \in \{T, U, R\}} P_g \log \frac{P_g}{\hat{P}_g} + P_T \text{KL}(p(\cdot|T) \| q) \\ &\quad + P_U \text{KL}(p(\cdot|U) \| \text{Unif}(U)) + P_R \text{KL}(p(\cdot|R) \| \text{Unif}(R)), \end{aligned} \quad (35)$$

where  $P_T \text{KL}(p(\cdot|T) \| q)$  can be assumed to be zero when we select exact top- $k$  in each row. The first term corresponds to group mass mismatch, while the remaining terms arise from the uniform values inside groups. We notice that for any set  $S$  with  $|S|$  elements, it holds  $0 \leq \text{KL}(r \| \text{Unif}(S)) \leq \log |S|$ . Thus, we have  $0 \leq P_U \text{KL}(p(\cdot|U) \| \text{Unif}(U)) \leq P_U \log k$  and  $0 \leq P_R \text{KL}(p(\cdot|R) \| \text{Unif}(R)) \leq P_R \log(N-2k)$ . Hence, we can yield the value range of the KL-divergence between the true attention and our attention in each row

$$\text{KL}(p \|\hat{p}) \leq \sum_{g \in \{T, U, R\}} P_g \log \frac{P_g}{\hat{P}_g} + P_U \log k + P_R \log(N-2k). \quad (36)$$

Assume attention weights follow a truncated power law  $p_{(i)} = \frac{i^{-\alpha}}{H_{N,\alpha}}$ ,  $\alpha > 1$  where  $H_{N,\alpha} = \sum_{j=1}^N j^{-\alpha}$ . Using integral inequality for

$f(x) = x^{-\alpha}$  ( $\alpha > 1$ ), we have the bound for  $H_{N,\alpha}$ :

$$\frac{1 - (N+1)^{1-\alpha}}{\alpha-1} \leq H_{N,\alpha} \leq 1 + \frac{1 - N^{1-\alpha}}{\alpha-1}. \quad (37)$$

Thus we can define  $H_{\min} = \frac{1 - (N+1)^{1-\alpha}}{\alpha-1}$  and  $H_{\max} = 1 + \frac{1 - N^{1-\alpha}}{\alpha-1}$ . Similarly, we can derive the bound for group sums. For  $S(a, b) = \sum_{i=a}^b i^{-\alpha}$ , we have  $\int_a^{b+1} x^{-\alpha} dx \leq S(a, b) \leq \int_{a-1}^b x^{-\alpha} dx$ . Hence, we have ranges for the three groups, respectively:

$$\frac{1 - (k+1)^{1-\alpha}}{\alpha-1} \leq S_T \leq 1 + \frac{1 - k^{1-\alpha}}{\alpha-1}, \quad (38)$$

$$\frac{(k+1)^{1-\alpha} - (2k+1)^{1-\alpha}}{\alpha-1} \leq S_U \leq \frac{k^{1-\alpha} - (2k)^{1-\alpha}}{\alpha-1} \quad (39)$$

$$\frac{(2k+1)^{1-\alpha} - (N+1)^{1-\alpha}}{\alpha-1} \leq S_R \leq \frac{(2k)^{1-\alpha} - N^{1-\alpha}}{\alpha-1} \quad (40)$$

The probabilities of the three groups satisfy  $P_T = \frac{S_T}{H_{N,\alpha}}$ ,  $P_U = \frac{S_U}{H_{N,\alpha}}$ ,  $P_R = \frac{S_R}{H_{N,\alpha}}$ . Thus, they can be bounded with  $\frac{S_{g,\min}}{H_{\max}} \leq P_g \leq \frac{S_{g,\max}}{H_{\min}}$ . Let  $x$  be the top- $k$  vector inserted into the index  $I$  and  $x_{nn}$  its retrieved neighbor, with distance  $d = \|x - x_{nn}\|$ . Assume group mass functions satisfy a Lipschitz condition:  $|\hat{P}_g(x) - \hat{P}_g(y)| \leq L_g \|x - y\|$  where  $L_g$  is the Lipschitz constant. Then we have  $|\hat{P}_g - P_g| \leq L_g d$  and relative error satisfies  $|\varepsilon_g| = \left| \frac{\hat{P}_g - P_g}{P_g} \right| \leq \frac{L_g}{P_g} d$ . The mass contribution to the KL divergence is defined as  $\text{KL}_{\text{mass}} = \sum_{g \in \{T, U, R\}} P_g \log \frac{P_g}{\hat{P}_g}$ . Substituting the relative error, we yield the exact identity  $\text{KL}_{\text{mass}} = \sum_g P_g \log \frac{1}{1 + \varepsilon_g} = -\sum_g P_g \log(1 + \varepsilon_g)$ . For  $|\varepsilon| < 1$ , the logarithm admits the Taylor expansion  $-\log(1 + \varepsilon) = -\varepsilon + \frac{\varepsilon^2}{2} + O(\varepsilon^3)$ . Substituting this expansion gives

$$\text{KL}_{\text{mass}} = -\sum_g P_g \varepsilon_g + \frac{1}{2} \sum_g P_g \varepsilon_g^2 + \sum_g P_g O(\varepsilon_g^3). \quad (41)$$

Both true and approximate block masses sum to one:  $P_T + P_U + P_R = 1$  and  $\hat{P}_T + \hat{P}_U + \hat{P}_R = 1$ . Using  $\hat{P}_g = P_g(1 + \varepsilon_g)$ , we can achieve  $\sum_g (\hat{P}_g - P_g) = 0$  which means  $\sum_g P_g \varepsilon_g = 0$ . Therefore, the first-order term vanishes, leaving

$$\text{KL}_{\text{mass}} = \frac{1}{2} \sum_g P_g \varepsilon_g^2 + \sum_g P_g O(\varepsilon_g^3). \quad (42)$$

When errors  $\varepsilon_g$  are small, cubic terms can be neglected. Under these conditions, the KL increase due to nearest neighbor retrieval error is approximately quadratic in group mass perturbations. Considering the index dimension  $k$  we leverage for ANN search, the optimal distance  $d_{nn}$  between the query and its ANN has a theoretical model [14]:

$$\mathbb{E}[d_{nn}] = c \cdot n^{-\frac{1}{k}} \quad (43)$$

where  $n$  denotes the sample number and  $C$  is a constant. In practice, we use ANN search to reduce the computational cost and achieve distance  $d = \rho d_{nn}$  where  $\rho \geq 1$ . Assume the distribution of  $d$  is sufficiently concentrated, we have  $\mathbb{E}[d^2] = \rho^2 c^2 D^{-\frac{2}{k}}$  where  $D$  is the index size. Accordingly, we can rewrite the group mass as

$$\begin{aligned} \mathbb{E}[\text{KL}_{\text{mass}}] &= \frac{1}{2} \mathbb{E}[d^2] \left( \frac{L_T^2}{P_T} + \frac{L_U^2}{P_U} + \frac{L_R^2}{P_R} \right) \\ &= C \rho^2 D^{-\frac{2}{k}} \left( \frac{L_T^2}{P_T} + \frac{L_U^2}{P_U} + \frac{L_R^2}{P_R} \right) \end{aligned} \quad (44)$$

where  $\rho$  denotes the quality of ANN result and  $C$  fuses the previous constants. Combining all components, it yields

$$\begin{aligned} \mathbb{E}[\text{KL}(p||\hat{p})] &\leq \mathbb{E}[\text{KL}_{\text{mass}}] + \frac{S_{U,\text{max}}}{H_{\text{min}}} \log k + \frac{S_{R,\text{max}}}{H_{\text{min}}} \log(N-2k) \\ &\xrightarrow{N \rightarrow \infty} C\alpha\rho^2 D^{-\frac{2}{k}} \left( \frac{L_T^2}{1 - (k+1)^{1-\alpha}} + \frac{L_U^2}{(k+1)^{1-\alpha} - (2k+1)^{1-\alpha}} \right. \\ &\quad \left. + \frac{L_R^2}{(2k+1)^{1-\alpha}} \right) + (k^{1-\alpha} - (2k)^{1-\alpha}) \log k + (2k)^{1-\alpha} \log(N-2k) \\ &\xrightarrow{D \rightarrow \infty} (k^{1-\alpha} - (2k)^{1-\alpha}) \log k + (2k)^{1-\alpha} \log(N-2k) := F(k) \end{aligned} \quad (45)$$

We can calculate the extreme point of  $F(k)$  when we take  $k = O(\log N)$  and find that the point is far smaller than one. In this case, we observe that the deviation of  $F(k)$  is dominated by the term  $\log N \cdot k^{-\alpha}$  and thus we have

$$\mathbb{E}[\text{KL}(p||\hat{p})] = \Theta(\log N \cdot k^{1-\alpha}). \quad (46)$$

Noted that if we extend the proof to the situation where the approximate histogram has  $T$  buckets, the approximation of  $\mathbb{E}(\text{KL}_{\text{mass}})$  in Inequality 45 still holds, and we have

$$F(k) = \sum_{i=1}^t (ik^{1-\alpha} - ((i+1)k)^{1-\alpha}) \log k + ((t+1)k)^{1-\alpha} \log(N - (t+1)k) \quad (47)$$

Accordingly, the deviation of  $F(k)$  can still be dominated by the term  $\log N \cdot k^{-\alpha}$  under the same parameter settings and the final expectation can be expressed as:

$$\mathbb{E}[\text{KL}(p||\hat{p})] = \Theta(\log N \cdot k^{1-\alpha}) \quad (48)$$

when  $T, k \ll N$ . According to our proof, the KL divergence of directly filling the mean value into the remaining  $N - k$  positions is finally dominated by the term  $k^{1-\alpha} \log(N - k)$ . Thus, the difference between our method and the method with  $T = 1$  can be written as  $\Delta \mathbb{E}[\text{KL}]$

$$\begin{aligned} &\approx (k^{1-\alpha} - (2k)^{1-\alpha}) \log k + (2k)^{1-\alpha} \log(N - 2k) - k^{1-\alpha} \log(N - k) \\ &= (k^{1-\alpha} - (2k)^{1-\alpha}) \log \frac{k}{N - k} + (2k)^{1-\alpha} \log \frac{N - 2k}{N - k} < 0, \end{aligned} \quad (49)$$

which validates our perspective that a  $T = 2$  method can better fit the original attention distribution.

## A.8 Proof of Theorem: Expectation of KL Divergence Comparison

**THEOREM A.5 (EXPECTATION OF KL DIVERGENCE COMPARISON).** Denote  $\text{KL}_{\text{mass}}$  to be the KL divergence that comes from mismatching the total probability mass assigned to each column subset, which is independent of how probabilities are distributed inside the subsets. Let  $\text{KL}_{\text{mass}}^{\text{skewed}}$  and  $\text{KL}_{\text{mass}}^{\text{flat}}$  denote  $\text{KL}_{\text{mass}}$  over attention rows with skewed and flat search vectors in the same  $A_{\text{head}}$ , respectively. Under the same second-order approximation [41], the following holds

$$\mathbb{E}[\text{KL}_{\text{mass}}^{\text{skewed}}] \leq \mathbb{E}[\text{KL}_{\text{mass}}^{\text{flat}}]. \quad (50)$$

**PROOF.** We follow the signs in the proof of Theorem 6.1. Instead of using nearest neighbor retrieval, assume we approximate block masses by a single global mean vector  $\hat{P}_T = \bar{P}_T, \hat{P}_U = \bar{P}_U, \hat{P}_R = \bar{P}_R$ ,

where  $\bar{P}_T = \bar{s}_{\text{nor}}, \bar{P}_U = k\bar{b}_1, \bar{P}_R = (N - 2k)\bar{b}_2$ . The resulting KL divergence of the new group mass can be rewritten as  $\mathbb{E}[\text{KL}_{\text{mass}}^{\text{mean}}] = \sum_{g \in \{T, U, R\}} P_g \log \frac{P_g}{\bar{P}_g}$ . Importantly, the remaining terms in Equation 35 are unchanged whether we use nearest neighbors or a global mean, since it only depends on uniformization within blocks. Let the global mean be  $\mu_g := \mathbb{E}[P_g]$ , and assume the constant approximation uses  $\bar{P}_g = \mu_g$ . We can write deviations as  $P_g = \mu_g + \delta_g, \mathbb{E}[\delta_g] = 0, \sum_g \delta_g = 0$ . For small deviations  $|\delta_g| \ll \mu_g$ , we have  $P_g \log \frac{P_g}{\mu_g} = \delta_g + \frac{\delta_g^2}{2\mu_g} + O(|\delta_g|^3/\mu_g^2)$ . After taking the expectation over the dataset, it yields:

$$\mathbb{E}[\text{KL}_{\text{mass}}^{\text{mean}}] = \frac{1}{2} \sum_g \frac{\text{Var}(P_g)}{\mu_g}. \quad (51)$$

Thus, the expected KL penalty from using a global mean is governed by the variance of block masses across queries. If nearest neighbor retrieval provides a conditional estimate of block masses, we have  $\hat{P}_g(x) \approx \mathbb{E}[P_g | x]$  for each skewed query  $x$ , and then the corresponding expected mass KL satisfies the approximation:

$$\mathbb{E}[\text{KL}_{\text{mass}}^{\text{NN}}] \approx \frac{1}{2} \sum_g \mathbb{E} \left[ \frac{\text{Var}(P_g | x)}{\mathbb{E}[P_g | x]} \right] \quad (52)$$

By the law of total variance, we have

$$\text{Var}(P_g) = \mathbb{E}[\text{Var}(P_g | x)] + \text{Var}(\mathbb{E}[P_g | x]),$$

which implies that conditioning on  $x$  typically reduces the residual variance. Hence, under our effective retrieval and the same second-order approximation, we have  $\mathbb{E}[\text{KL}_{\text{mass}}^{\text{NN}}] \leq \mathbb{E}[\text{KL}_{\text{mass}}^{\text{mean}}]$ .  $\square$

## A.9 Proof of Theorem: Bounded Error of Contribution Ratio Estimation

**THEOREM A.6 (BOUNDED ERROR OF CONTRIBUTION RATIO ESTIMATION).** Let  $\tilde{y} = \tilde{y}^{(B)} + \tilde{y}^{(B \rightarrow A)}$  be the exact decomposition in Eq. (19), and define  $\rho_A$  and  $\hat{\rho}_A$  by Eq. (20) and Eq. (11), respectively. Then the estimator  $\hat{\rho}_A$  satisfies the bound

$$|\hat{\rho}_A - \rho_A| \leq \frac{\alpha_A(1 - \rho_A) + \beta_B \rho_A}{\beta_A \rho_A + \alpha_B(1 - \rho_A)}. \quad (53)$$

where  $\alpha_A, \alpha_B$  quantify the alignment with the Value-mean baseline term, and  $\beta_A, \beta_B$  quantify the alignment with the cross-modal attention term.

**PROOF.** We assume the following conditions based on Cauchy-Schwarz inequality: there exist  $\alpha_A, \alpha_B, \beta_A, \beta_B \in [0, 1]$ ,

$$|a^\top \tilde{y}^{(B)}| = \alpha_A \|\tilde{y}^{(B)}\|_2, \quad |b^\top \tilde{y}^{(B)}| = \alpha_B \|\tilde{y}^{(B)}\|_2, \quad (54)$$

$$|a^\top \tilde{y}^{(B \rightarrow A)}| = \beta_A \|\tilde{y}^{(B \rightarrow A)}\|_2, \quad |b^\top \tilde{y}^{(B \rightarrow A)}| = \beta_B \|\tilde{y}^{(B \rightarrow A)}\|_2. \quad (55)$$

Here  $\alpha_A, \alpha_B$  quantify the alignment with the Value-mean baseline term, and  $\beta_A, \beta_B$  quantify the alignment with the cross-modal attention term. Let

$$u := \|\tilde{y}^{(B \rightarrow A)}\|_2, \quad v := \|\tilde{y}^{(B)}\|_2,$$

so that  $\rho_A = \frac{u}{u+v}$  and  $\tilde{y} = \tilde{y}^{(B \rightarrow A)} + \tilde{y}^{(B)}$ .

We first bound the numerator and denominator of  $\hat{\rho}_A$ . By the triangle inequality,

$$|a^\top \tilde{y}| = |a^\top \tilde{y}^{(B \rightarrow A)} + a^\top \tilde{y}^{(B)}| \geq |a^\top \tilde{y}^{(B \rightarrow A)}| - |a^\top \tilde{y}^{(B)}|,$$

and similarly

$$|a^\top \tilde{y}| \leq |a^\top \tilde{y}^{(B \rightarrow A)}| + |a^\top \tilde{y}^{(B)}|.$$

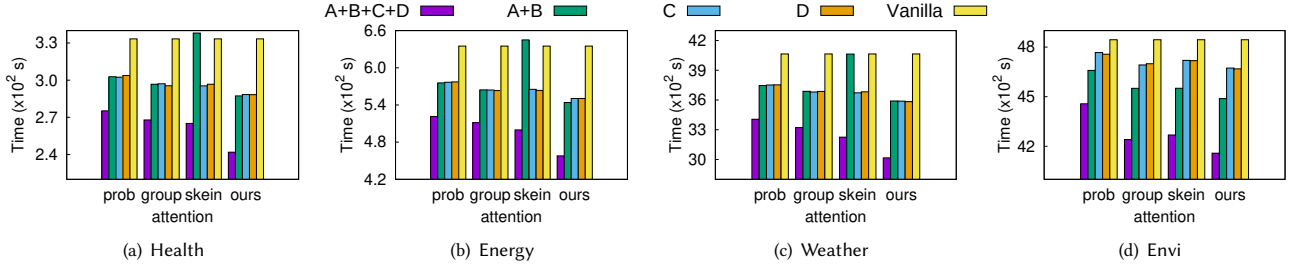


Figure 18: Training time of Aurora accelerated with different cross-modal blocks on 4 datasets.

Applying the assumptions gives

$$\beta_{Au} - \alpha_{Av} \leq |a^\top \tilde{y}| \leq \beta_{Au} + \alpha_{Av}.$$

By the same argument, we obtain

$$\alpha_{Bv} - \beta_{Bu} \leq |b^\top \tilde{y}| \leq \alpha_{Bv} + \beta_{Bu}.$$

Therefore, the ratio

$$\hat{\rho}_A = \frac{|a^\top \tilde{y}|}{|a^\top \tilde{y}| + |b^\top \tilde{y}|}$$

is bounded between the worst-case ratios obtained by taking the smallest possible numerator and largest possible denominator, and vice versa. In particular,

$$\hat{\rho}_A \geq \frac{\beta_{Au} - \alpha_{Av}}{(\beta_{Au} + \alpha_{Av}) + (\alpha_{Bv} + \beta_{Bu})},$$

and

$$\hat{\rho}_A \leq \frac{\beta_{Au} + \alpha_{Av}}{(\beta_{Au} + \alpha_{Av}) + (\alpha_{Bv} - \beta_{Bu})}.$$

Now recall that  $\rho_A = \frac{u}{u+v}$ , i.e.,  $u = \rho_A(u+v)$  and  $v = (1-\rho_A)(u+v)$ . Substituting into the above bounds and simplifying yields

$$|\hat{\rho}_A - \rho_A| \leq \frac{\alpha_A(1-\rho_A) + \beta_B\rho_A}{\beta_A\rho_A + \alpha_B(1-\rho_A)},$$

which completes the proof.  $\square$

## A.10 Proof of Corollary: Concentration of Batch-wise Estimator

**COROLLARY A.7 (CONCENTRATION OF BATCH-WISE ESTIMATOR).** Assume the estimator uses the batch-wise  $S_A := \frac{1}{B} \sum_{i=1}^B \hat{\rho}_A(i)$ . Each  $\hat{\rho}_A(i) \in [0, 1]$  denotes i.i.d. samples. Then for any  $\delta \in (0, 1)$ , with probability at least  $1 - \delta$ ,

$$|S_A - \mathbb{E}[\hat{\rho}_A]| \leq \sqrt{\frac{\ln(2/\delta)}{2B}}. \quad (56)$$

**PROOF.** Since each  $\hat{\rho}_A(i) \in [0, 1]$ , by Hoeffding's inequality for bounded independent random variables, for any  $\epsilon > 0$  we have

$$\Pr\left(\left|\frac{1}{B} \sum_{i=1}^B \hat{\rho}_A(i) - \mathbb{E}[\hat{\rho}_A]\right| \geq \epsilon\right) \leq 2 \exp(-2B\epsilon^2).$$

Setting the right-hand side to  $\delta$  and solving for  $\epsilon$  gives

$$\epsilon = \sqrt{\frac{\ln(2/\delta)}{2B}}.$$

Therefore, with probability at least  $1 - \delta$ ,

$$|S_A - \mathbb{E}[\hat{\rho}_A]| \leq \sqrt{\frac{\ln(2/\delta)}{2B}}. \quad \square$$

## A.11 Experiments on Efficiency Varying Accelerated Blocks

We report the running time of Aurora, accelerating each cross-modal attention block with our proposed attention mechanism in Figure 18. Since the input lengths of block A and B are both  $N_1$  and  $N_2$ , we thus combine them and calculate the time together. The predicted length of Health and Energy is 12, while the length of Weather and Envi is 48. Across all four datasets, LEO consistently achieves the lowest training time under the same model settings and input lengths. When applying our attention mechanism on individual blocks, for long-sequence settings (Weather and Envi), each block with LEO yields a modest but stable speedup of approximately 3–4% over the complete model, indicating effective efficiency gains without degradation. The advantage becomes slightly more pronounced in short-sequence scenarios (Health and Energy), where ours reduces runtime by about 5–6%, demonstrating good scalability across different sequence lengths. Notably, competing structured approaches (e.g., Group and Skein) show less consistent behavior across datasets, whereas our method maintains uniformly strong performance. Overall, these results confirm that the proposed sparsification strategy introduces reliable computational savings and scales effectively across diverse sequence lengths and workloads.

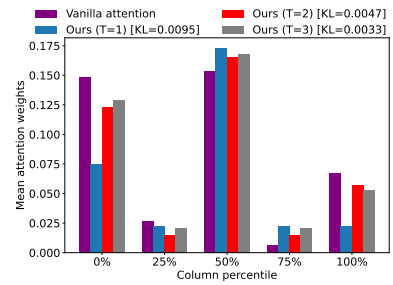


Figure 19: Example of approximate attention varying approximate histogram buckets  $T$ .

## A.12 Example of Our Attention Varying Histogram Buckets $T$

We present an example of approximate attention varying  $T$  with input length of 32 in Figure 19, showing that the KL divergence improvement between  $T = 1$  and  $T = 2$  is far more than that between  $T = 2$  and  $T = 3$ . Specifically, the attention weights are very close among different methods when treating large weights like the 50% percentile in the figure. However, the weights at the 0%, 100% percentile vary significantly, demonstrating the better approximation

with larger  $T$ . Meanwhile, a larger  $T$  brings more computational cost with little improvement on the KL divergence, indicating that the bucket number  $T$  should be controlled. In practice, setting  $T = 2$  is enough.

### A.13 Experiments on MSE Varying Input Lengths.

Here we report the MSE on Health varying input lengths  $N_3$  and  $N_4$  in Figure 20. We can conclude the same as we argue on  $N_1$  and  $N_2$ , indicating the degradation of prediction MSE when scaling.

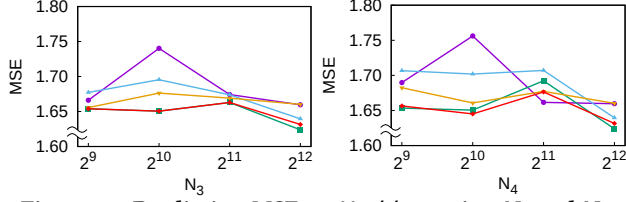


Figure 20: Prediction MSE on Health varying  $N_3$  and  $N_4$ .

### A.14 Ablation Study

Here, we report an ablation study of CSS with approximate histograms and Query matrix pruning on Envi in Figure 21 and Figure 22, which reaches the same conclusion as on Health.

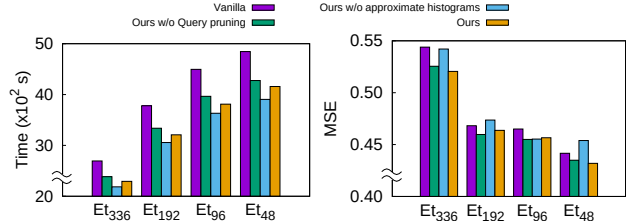


Figure 21: Prediction MSE and training time of ablation on the CSS with approximate histograms on Envi.

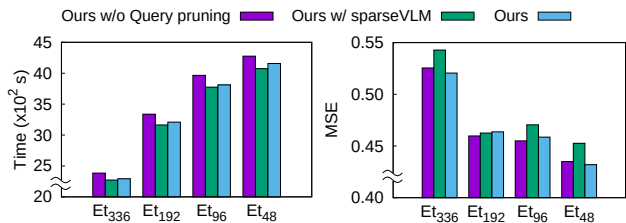


Figure 22: Prediction MSE and training time of ablation on Query matrix pruning on Envi.

### A.15 Visualization of the Loss Curve

Here we present the training and validation loss curve during training on Energy with a prediction length of 12 in Figure 23, which shows Aurora can converge with our proposed attention.

### A.16 Visualization of Training Loss with Fixed $k$

Here, we present the training loss curve with fixed  $k$  and adaptive  $k$  with Algorithm 7 on Health with a prediction length of 48 in Figure 24. For fairness, we set a fixed  $k$  the same as  $k_{\max}$  for the adaptive method. The figure shows that the training losses of the two methods are similar, and the adaptive method achieves shorter

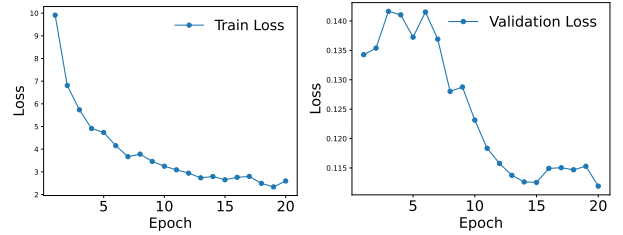


Figure 23: Training and validation loss curve on Energy dataset

training time, indicating that Algorithm 7 can dynamically adjust  $k$  for different inputs.

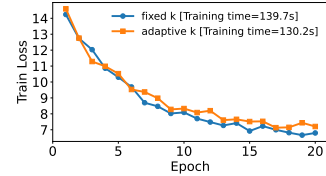


Figure 24: Training loss comparison between fixed  $k$  and adaptive  $k$ .

### A.17 Hyperparameter for Training

Table 4 summarizes the training hyperparameters used across all datasets. Besides, we set  $h = 0.9$ ,  $\tau = 0.1$  and  $\theta_0 = 0.05$  in our experiments.

Table 4: Hyperparameters

| Dataset name | Batch size | Epochs |
|--------------|------------|--------|
| Health       | 16         | 20     |
| Energy       | 16         | 20     |
| Weather      | 16         | 20     |
| Environment  | 4          | 10     |

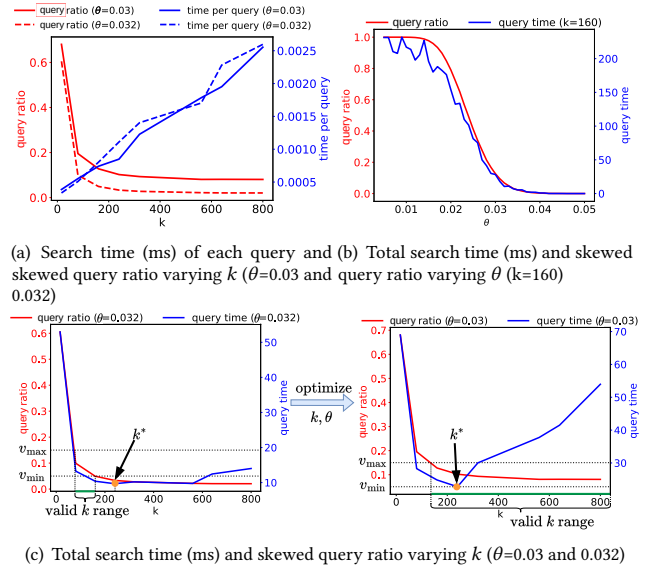
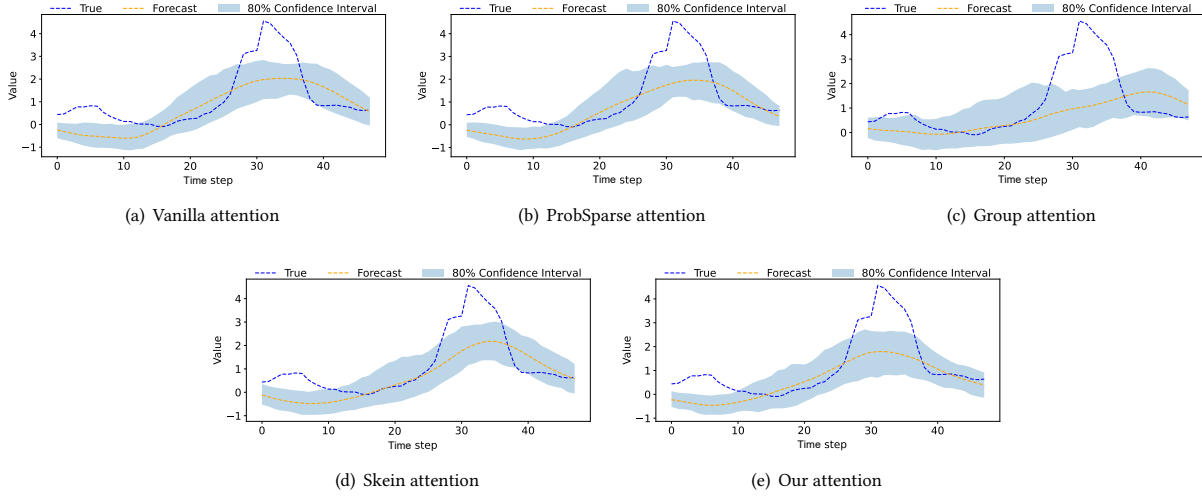


Figure 25: An example of the relationship between the search time and skewed query ratio ( $M = N = 2^{11}$ ) with synthetic data. In practice,  $k^*$  varies with different  $\theta$  under the same input length.



**Figure 26: Visualization of prediction results on Health**

### A.18 Visualization of the Prediction Result on Health

Here, we present a visualization of the predictions for a prediction length of 48 on Health across five attention methods in Figure 26. The figures show that our attention method can effectively predict the peak at about time step 31, which is consistent with the original

trend and more accurate than the other baselines. Additionally, although the 80% confidence interval of Group covers the most time steps of the true values, it predicts the peak at the wrong step with relatively large error. Therefore, our method can provide an accurate prediction of the periodic pattern validated by our experiment results.